



# Type-Constrained Code Generation with Language Models

NIELS MÜNDLER\*, ETH Zurich, Switzerland

JINGXUAN HE\*, University of California at Berkeley, USA

HAO WANG, University of California at Berkeley, USA

KOUSHIK SEN, University of California at Berkeley, USA

DAWN SONG, University of California at Berkeley, USA

MARTIN VECHEV, ETH Zurich, Switzerland

Large language models (LLMs) have achieved notable success in code generation. However, they still frequently produce uncompileable output because their next-token inference procedure does not model formal aspects of code. Although constrained decoding is a promising approach to alleviate this issue, it has only been applied to handle either domain-specific languages or syntactic features of general-purpose programming languages. However, LLMs frequently generate code with typing errors, which are beyond the domain of syntax and generally hard to adequately constrain. To address this challenge, we introduce a type-constrained decoding approach that leverages type systems to guide code generation. For this purpose, we develop novel prefix automata and a search over inhabitable types, forming a sound approach to enforce well-typedness on LLM-generated code. We formalize our approach on a foundational simply-typed language and extend it to TypeScript to demonstrate practicality. Our evaluation on the HumanEval and MBPP datasets shows that our approach reduces compilation errors by more than half and significantly increases functional correctness in code synthesis, translation, and repair tasks across LLMs of various sizes and model families, including state-of-the-art open-weight models with more than 30B parameters. The results demonstrate the generality and effectiveness of our approach in constraining LLM code generation with formal rules of type systems.

CCS Concepts: • **Theory of computation** → **Formal languages and automata theory**; • **Software and its engineering** → **General programming languages**; • **Computing methodologies** → **Machine learning**.

Additional Key Words and Phrases: Code Generation, Language Model, Type System, Program Synthesis, Program Translation, Program Repair, Constrained Decoding

## ACM Reference Format:

Niels Mündler, Jingxuan He, Hao Wang, Koushik Sen, Dawn Song, and Martin Vechev. 2025. Type-Constrained Code Generation with Language Models. *Proc. ACM Program. Lang.* 9, PLDI, Article 171 (June 2025), 26 pages. <https://doi.org/10.1145/3729274>

## 1 Introduction

Large language models (LLMs) are remarkably successful in diverse fields [12, 27, 49] and increasingly used in everyday coding tasks [25, 67]. They show promising capabilities at synthesizing code from natural language descriptions [37, 58], translating between programming languages [58], and repairing incorrect programs [44, 73]. Despite these achievements, LLM-generated code often

\*Both authors co-lead this project.

Authors' Contact Information: [Niels Mündler](#), ETH Zurich, Zurich, Switzerland, [niels.muendler@inf.ethz.ch](mailto:niels.muendler@inf.ethz.ch); [Jingxuan He](#), University of California at Berkeley, Berkeley, USA, [jingxuan.he@berkeley.edu](mailto:jingxuan.he@berkeley.edu); [Hao Wang](#), University of California at Berkeley, Berkeley, USA, [hwang628@berkeley.edu](mailto:hwang628@berkeley.edu); [Koushik Sen](#), University of California at Berkeley, Berkeley, USA, [ksen@berkeley.edu](mailto:ksen@berkeley.edu); [Dawn Song](#), University of California at Berkeley, Berkeley, USA, [dawnsong@berkeley.edu](mailto:dawnsong@berkeley.edu); [Martin Vechev](#), ETH Zurich, Zurich, Switzerland, [martin.vechev@inf.ethz.ch](mailto:martin.vechev@inf.ethz.ch).



This work is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 2475-1421/2025/6-ART171

<https://doi.org/10.1145/3729274>

contains compilation errors, logic flaws, or security vulnerabilities [20, 52, 54]. These issues arise because LLMs generate code by iteratively sampling the next token from a vocabulary of tokens – a probabilistic process that does not provide any formal guarantees.

A promising technique to address this limitation is *constrained decoding*, which enforces the formal rules of programming languages during LLMs’ code generation process, rejecting invalid tokens and ensuring only valid tokens are considered as generation candidates. Previous studies have shown that constrained decoding improves adherence to program syntax [8, 41, 56, 65]. However, these improvements are limited, as syntax accounts for only a small part of overall program correctness. For instance, in our evaluation of state-of-the-art open-weight LLMs (§5), syntactic errors make up on average 6% of all compilation errors in generated TypeScript code.

**Key Challenge: Generating Well-Typed Code.** Beyond program syntax, type systems detect and reject bugs at compile time [40, 43] and are therefore enforced in many popular programming languages [4, 10, 19]. We observe that LLMs struggle to generate well-typed code [20, 29, 62], as typing rules significantly complicate the generation of valid code [61]. In our evaluation of LLMs (§5), on average 94% of compilation errors result from failing type checks. This suggests a promising direction: guiding LLMs’ code generation process by incorporating the formal rules of type systems.

However, implementing this approach is challenging because type systems can in general not be captured by context-free grammars [43], prohibiting the application of prior constrained decoding methods developed for program syntax [8, 65]. Furthermore, besides deriving and maintaining a type environment for completed expressions during generation (similar to classic type systems), we need to accurately assess and handle partial expressions. Specifically, for each currently generated partial expression, we must decide whether the partial expression can be completed to match a required type. Determining this would allow us to constrain the LLM to provably generate well-typed expressions upon termination, but involves solving the challenging problem of type inhabitation [30, 66] in the novel context of LLM-based code generation.

**This Work: Type-Constrained Decoding.** In this work, we introduce *type-constrained decoding*<sup>1,2</sup>, addressing the challenge of generating well-typed code using LLMs. We develop a sound algorithm to determine if a partial program can be completed into a well-typed program. This algorithm is based on a novel non-deterministic automaton we construct. The automaton incrementally builds abstract syntax trees described by the partial program and annotates them with type-relevant context, e.g., declared identifiers and expression types. It leverages such information to maintain a *prefix property*, ensuring that parsing a program prefix only results in a non-empty set of states when it can be completed into a well-typed program. To guarantee the prefix property, we design a sound type search algorithm that determines whether a partial expression can inhabit a given type. We construct our automaton for a generic, simply-typed Turing-complete calculus [10].

To demonstrate its practical effectiveness, we instantiate our approach on a non-trivial subset of TypeScript. We choose TypeScript for three key reasons: (i) it is currently one of the most actively used languages, e.g., in open-source projects on GitHub [26, 38]; (ii) as we show, state-of-the-art LLMs fail to reliably generate well-typed TypeScript code; (iii) its core type system is simple enough [10] to be suitable for developing the first prototype of our approach. We perform a comprehensive evaluation on TypeScript versions of the widely-used HumanEval and MBPP benchmarks [5, 13, 14], focusing on three common coding tasks: synthesis, translation, and repair. Our experimental results show that type-constrained decoding significantly enhances code generation for LLMs of various sizes (2B-34B parameters). For synthesis and translation, it reduces

<sup>1</sup>Our code implementation is publicly available at <https://github.com/eth-sri/type-constrained-code-generation>.

<sup>2</sup>A full version of our paper with appendices can be found at <https://arxiv.org/abs/2504.09246>.

compilation errors by more than half and increases functional correctness relatively by 3.5% to 5.5%. Additionally, it enhances functionally correct repair of non-compiling code relatively by 37% on average. We further investigate our approach in depth through a runtime analyses and case studies.

We highlight that our type constraining approach is broadly applicable to any language derivable from the core calculus, any code generation task in these languages, and any LLM utilizing next-token generation. In §6, we envision how our approach can benefit other production-ready languages and closed-weight LLMs.

**Main Contributions.** Our main contributions can be summarized as follows:

- A prefix automaton and a type search algorithm to enable type constraining for LLM-based code generation, demonstrated on a generic, simply-typed core calculus (§3).
- An instantiation and extension of our approach to the popular TypeScript language (§4).
- An extensive evaluation across various LLMs and coding tasks, showing the significant benefit of our approach in reducing compilation errors and increasing functional correctness (§5).

## 2 Background and Overview

In this section, we first provide relevant background on LLM-based code generation and constrained decoding. Then, we motivate our type constraining approach using an illustrative example and present a high-level overview of its construction.

### 2.1 Background on LLM-Based Code Generation and Constrained Decoding

**LLM-Based Code Generation.** LLMs generate code incrementally by sampling one token at a time in an iterative manner, as depicted in Algorithm 1 (without the blue highlights). A user prompt  $x$  specifies a code generation task for a trained LLM. At Line 1, the output program  $s$  is initialized to an empty string or a program prefix provided in  $x$ , e.g., a function signature. At the beginning of each generation iteration (Line 3), the LLM takes as input a concatenation  $x \circ s$  of the prompt  $x$  and the current partial program  $s$ . It then predicts a probability distribution  $v$  over a fixed, finite set of *tokens*, the *vocabulary*, where each token may be a single Unicode character or a string of multiple characters. All common singleton characters are included in LLMs' vocabulary, ensuring that any standard program can be produced by concatenating tokens [59]. Next, based on distribution

$v$ , a token  $t$  is sampled (Line 5) and appended to the program  $s$  (Line 10). This process is repeated until we encounter the special token *EOS* which signifies the end of the sequence (Line 9).

LLMs learn to predict adequate probability distributions from extensive training on natural and programming languages [12, 58, 72]. These distributions implicitly encode language rules, allowing LLMs to successfully solve code generation tasks [13, 28, 58]. However, LLMs may fail to infer complex rules [9, 21, 71], derive incomplete rules for less common languages [13, 51], and, due to the probabilistic nature of its generation procedure, not consistently follow formal language rules.

---

**Algorithm 1** Vanilla LLM-based code generation (without the blue highlights) vs. constrained decoding (with the blue highlights)

---

**Input:** LLM, prompt  $x$ , completion engine  $CE_L$  for language  $L$

**Output:** Program  $s$  such that  $s \in L$

```

1: initialize  $s$ 
2: while true do
3:    $v := \text{LLM}(x \circ s)$ 
4:   while true do
5:      $t \sim v$ 
6:     if  $CE_L(s \circ t)$  then break
7:     elif  $t = \text{EOS}$  and  $s \in L$  then break
8:     else  $v[t] := 0$ ; normalize  $v$ 
9:   if  $t = \text{EOS}$  then break
10:   $s := s \circ t$ 
11: return  $s$ 

```

---

|                                                 | <completion>     | Vanilla | Syntax | Types  | Description           |
|-------------------------------------------------|------------------|---------|--------|--------|-----------------------|
| <b>function</b> is_int(text: string): boolean { | (1) ;            | accept  | reject | reject | Syntactically invalid |
| <b>const</b> num = Number(text);                | (2) ber          | accept  | accept | reject | Undeclared identifier |
| <b>return</b> !isNaN(num) &&                    | (3) ()           | accept  | accept | reject | Disallowed operator   |
| parseInt(num <completion>                       | (4) , 10)        | accept  | accept | reject | Invalid argument type |
|                                                 | (5) .toString()) | accept  | accept | accept | Well-formed option    |

Figure 1. Left is a partial TypeScript program derived from instance #113 of the MBPP benchmark [5], awaiting completion. Right are five completion options: (1)–(4) are invalid and (5) is well-formed. Our type-constrained decoding is the only approach capable of correctly rejecting invalid completions and accepting the valid one.

**Constrained Decoding.** The aforementioned shortcoming of LLMs can be mitigated by employing *constrained decoding*, which analyzes the intermediate model outputs  $s$  during the generation process and enforces that only valid tokens are incorporated. Specifically, constrained decoding leverages a *completion engine*  $CE_L$ , specific to a language  $L$ . Computing  $CE_L(s)$  returns whether partial program  $s$  can be completed to a well-formed program in  $L$ , meaning whether there exists a (possibly empty) string  $s'$  such that  $s \circ s' \in L$ . Equivalently,  $CE_L(s)$  determines whether  $s$  belongs to the *prefix language*  $L^P$  of  $L$ , i.e., whether  $s \in L^P$ .  $L^P$  is formally defined as follows:

DEFINITION 1. For a given language  $L$ , its *prefix language* is  $L^P := \{s \mid \exists s' : s \circ s' \in L\}$ .

As illustrated in blue highlights of Algorithm 1, constrained decoding differs from vanilla LLM-based code generation by adding an additional sample-and-check loop at Line 4 around the token sampling process at Line 5. A sampled token  $t$  is considered further only if  $s \circ t$  can be completed to a well-formed program (Line 6) or  $t$  is *EOS* and  $s$  is already well-formed in  $L$  (Line 7). Otherwise, the probability of  $t$  is set to zero at Line 8, and the sample-and-check loop repeats. Note that a token  $t$  satisfying either Line 6 or Line 7 always exists, because  $s$  is in  $L^P$  and LLMs' vocabulary contains all common characters. Therefore, the number of iterations of the loop at Line 4 is bounded by the fixed LLM vocabulary size. In practice, only few iterations are needed (§5.3) and do not require additional LLM inference, ensuring a reasonable runtime overhead compared to vanilla decoding.

The token-level guarantees extend inductively to guarantee the final program's validity with respect to  $L$ . At Line 1, we start with a valid prefix in  $L^P$ , i.e., either an empty string or a valid prefix provided in the user prompt. The check at Line 6 ensures that all intermediate outputs  $s$  are prefixes in  $L^P$ . Additionally, Line 7 and Line 9 ensure that the return statement in Line 11 is reached only if  $s \in L$ . As an additional benefit, by steering previously ill-formed generations into well-formed ones, constrained decoding also increases the likelihood of generating functionally correct code.

Note that commonly used grammar and type checkers can not be used as a completion engine for constrained decoding. They judge whether a program string  $s$  is well-formed according to the language  $L$ , i.e., whether  $s \in L$ . When  $s$  is not a complete program in  $L$ , but a valid prefix in  $L^P$ , they return a different output than  $CE_L(s)$ , which is not suitable for use in Algorithm 1.

## 2.2 Overview of Our Type Constraining Approach

**Inadequacy of Syntax-Only Constraining.** To apply the constrained decoding algorithm described in §2, one needs to choose a language  $L$  and implement the completion engine  $CE_L$ . Recent work has explored defining  $L$  as the set of syntactically valid programs, thus leveraging the syntactic rules of programming languages for constrained decoding [8, 65, 70]. However, the benefits of this approach are limited, because syntax accounts for only a small portion of overall program correctness. For instance, across our evaluations (§5), only 3.5% of the functional errors and 6% of the compilation errors in LLM-generated code are due to syntactic errors.

We illustrate this limitation using the example in Figure 1. It presents five completion candidates for a partial program: (1)-(4) will lead to compilation errors and only (5) can result in a well-formed program. Based on syntax, completions that contain line terminations or invalid characters (e.g., \$) could be rejected (1). However, many other cases, including (2)-(4), do not break syntactic rules but still cause compilation errors. For instance, candidate (2) results in accessing an undeclared identifier. In candidate (3), the function call operator will fail at execution time, as `num` is a number and can not be called. Candidate (4) passes a value of unexpected format to `parseInt`, which expects the first argument to be a string. In this example, (4) is generated by CodeLlama 34B [58]. Syntax-only constraining accepts this invalid completion, leading to a non-compilable final output.

**Our Approach: Leveraging the Type System.** We require stronger constraints to effectively guide the model generation. Beyond syntax, type systems are commonly utilized in compilers, enforcing semantic rules to detect and reject bugs at compile time [23]. For Figure 1, the TypeScript type system would correctly reject code containing erroneous completions (2)-(4). Therefore, in this work, we propose leveraging type systems in constrained decoding to guide code generation. Our method accurately detects that only candidate (5) is a valid completion, guiding CodeLlama 34B to adopt this option and complete the program correctly. As detailed in §5, our experimental results demonstrate that our approach more than halves compiler errors in generated code and consistently increases the proportion of functionally correct programs.

Incorporating typing rules into code generation offers substantial potential but presents a significant challenge. Previous research has focused primarily on constrained decoding for context-free languages, for which prefixes can be efficiently determined [8, 65, 70]. Type systems, however, require language specifications that exceed the capabilities of context-free grammars [43], inhibiting the direct application of prior techniques to type-constrained decoding. Moreover, determining whether a partially generated expression can be completed to be a well-typed full expression involves not only type checking and inference, as done in traditional compilers, but also addressing type inhabitation [39, 66].

To address these challenges, we design and implement a practical approach to determine whether a string can be completed to a well-typed program. We begin by developing a specialized kind of non-deterministic automaton that maintains a *prefix property*, formally defined in §3.2. This property ensures that every reachable state can lead to an accepting state. We leverage this property to build a completion engine for constrained decoding as in Algorithm 1. We construct such a completion engine to enforce well-typedness for a simply-typed language  $L_B$  in §3.3–§3.5 and extend it to a core subset of TypeScript in §4. At a high level, the automaton acts as a syntactic parser, additionally maintaining information about initialized variables, enclosing function declarations, and other type-related aspects of the partially parsed syntax tree. This is possible through dynamically created annotated states that track the additional information.

In Figure 2, we provide a concrete example for our prefix automata. Every state represents the currently parsed syntactic component and additionally tracks the surrounding typing information. For example, after parsing the partial program in Figure 1, the automaton currently parses an expression as the first argument to function `parseInt`. Transitions are annotated with further code completions that are deemed admissible based on the syntax and typing information. In the first state, the automaton has parsed `num`, inferring from previous declarations that it represents an identifier of type number. Based on the signature of the `parseInt` function call, the required type of the completed

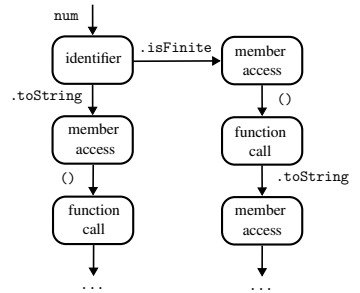


Figure 2. An example of a prefix automaton.



argument is `string`. The automaton now determines the admissible transitions from the identifier state. State transitions corresponding to completions (1)-(4) from Figure 1 are disallowed, as they are determined to violate type rules based on the tracked type information. Further, the automaton needs to determine which operations on the current expression `num` of type `number` can be applied to obtain an expression of type `string`. To achieve this, we develop a *type reachability search* algorithm, which finds `string`-typed expressions `num.toString()` and `num.isFinite().toString()`. Therefore, it returns that accesses to members `.toString` and `.isFinite` are admissible, resulting in the two depicted transitions with the corresponding labels. In our experiment, CodeLlama 34B chooses to transition along `.toString()`, the more likely completion based on its training data. Note that in our actual automaton formalism, as described at the end of §3.2, state transitions are on a character level. Figure 2 condenses character-level transitions into string-level transitions for presentation purposes.

The type reachability algorithm seeks to identify sequences of operators applied to a given expression such that the resulting expression possesses a required type. Conceptually, it performs a search over an abstracted type graph, whose nodes are types, and edges represent well-typed operations connecting the input and output types. An example of such a (partial) graph is shown in Figure 3, with a valid path highlighted in green color. Starting from the derived number type of `num`, the search first traverses a member access edge to reach the nullary function type `() => string`. Then, it traverses an edge representing a function call to reach the goal type `string`, concluding that the combination of traversed operators `.toString()` is a well-formed completion for Figure 1. The path for `num.isFinite().toString()` is analogously valid but omitted in Figure 3 for brevity. This type reachability search is invoked every time a partial expression is parsed, in order to determine valid transitions in the prefix automaton.

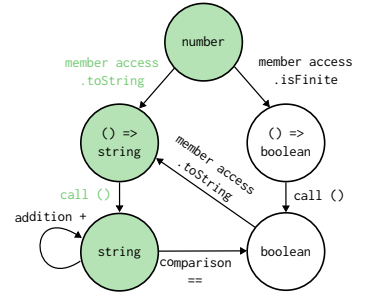


Figure 3. An example of a partial type search graph.

We implement our approach for a significant subset of TypeScript (§4) and experimentally evaluate it for various LLMs and three important code generation tasks: synthesis, translation, and repair (§5). The results demonstrate that our approach provides significant benefits in both reducing compilation errors for LLM-generated code and increasing their functional correctness.

### 3 Our Type Constraining Approach

In this section, we first present a generic, simply-typed language  $L_B$  (§3.1). Then, we present our type constraining approach using  $L_B$ . Specifically, we introduce our prefix automaton formalism (§3.2) and define increasingly complex automata for parsing well-typed fragments of  $L_B$ , beginning with identifiers, literals, and types (§3.3), continuing to expressions, including type search for type-restricted expressions (§3.4), and concluding with statements (§3.5).

#### 3.1 A Simply Typed Language

We define a simply typed, Turing-complete language,  $L_B$ . Its grammar and type system are generic, resembling the principles found in popular statically typed languages, such as TypeScript, Java, and Go. However, there may be a slight bias towards TypeScript, as our implementation is based on it.

**Syntax.** The syntax of  $L_B$  is shown in Figure 4. The language includes expressions, type-annotated variable and function definitions, and control flows. Overall, it is based on a core subset of TypeScript [10] but can be adapted for other statically typed languages. Similar to Bierman et al. [10], we represent Kleene-Star repetitions using an overline, e.g.,  $\bar{s}$  represents a sequence of statements  $s$ , and

|                           |                      |                                          |                        |
|---------------------------|----------------------|------------------------------------------|------------------------|
| $l ::=$                   | Literal              | $p ::= x : T$                            | Typed Identifier       |
| $\backslash d+$           | Numeric Literal      | $T ::=$                                  | Type                   |
| $"\backslash w *"$        | String Literal       | number                                   | Numeric Type           |
| $true \mid false$         | Boolean Literal      | string                                   | String Type            |
|                           |                      | boolean                                  | Boolean Type           |
| $x ::= \backslash w+$     | Identifier           | $(\bar{p}) \Rightarrow T$                | Function Type          |
| $e ::= e_0 \mid e_1$      | Expression           | $s ::=$                                  | Statement              |
| $e_0 ::=$                 | Base Expression      | let $x : T$ ;                            | Variable Declaration   |
| $l$                       | Literal              | $e$ ;                                    | Expression Statement   |
| $x$                       | Identifier           | return $e$ ;                             | Return Statement       |
| $(\bar{p}) \Rightarrow e$ | Function Expression  | $\{ \bar{s} \}$                          | Statement Block        |
| $(e)$                     | Grouped Expression   | function $x (\bar{p}) : T \{ \bar{s} \}$ | Function Definition    |
| $e_1 ::=$                 | Extension Expression | if $(e) s$ else $s$                      | If-Then-Else Statement |
| $e \odot e$               | Binary Operator      |                                          |                        |
| $e(\bar{e})$              | Function Call        |                                          |                        |
| $e.n$                     | Member Access        | $M ::= \bar{s}$                          | Program                |

Figure 4. The syntax of  $L_B$ . Expressions are categorized into base and extension expressions. The latter extends a given expression with suffix operators to form more complicated expressions.

adhere to the TypeScript documentation to annotate parameter types in function signatures with argument names [17]. We make a distinction between base and extension expressions. The latter applies operators to previous expressions, leading to more complex expressions. This differentiation is useful later in §3.4 for constructing the prefix automaton for parsing expressions.

**Expression Typing Rules.** The typing rules for  $L_B$ 's expressions are detailed in Figure 5. These rules form a subset of safeFTS, a type-safe portion of TypeScript described by Bierman et al. [10], allowing us to leverage their soundness results. The type rules for  $L_B$  use the standard concept of a *type environment*, denoted as  $\Gamma$ , which is a collection of pairs  $(x : T)$  of identifiers  $x$  and types  $T$ . We write  $\Gamma \vdash e : T$  if the expression  $e$  has type  $T$  in the type environment  $\Gamma$ . An expression  $e$  is considered valid if its type can be derived by applying the given typing rules.

Literals are evaluated to their respective types (LIT – {NUM, STR, BOOL}). Identifiers  $x$  are evaluated based on the corresponding type in the type environment (IDENT). Anonymous functions are typed according to their annotated parameter types, with the return type determined by the returned expression (ANON). Grouping preserves the type of the inner expression (GROUP).

Binary operators have predefined signatures  $S_1 \odot S_2 : T$ , such as  $\text{number} + \text{number} : \text{number}$  for addition and  $T = T : T$  for assignments. These signatures must be satisfied in well-typed expressions (OP). Function calls require parameters to match the function signature (CALL). The type of member accesses  $e.n$  is determined using an auxiliary function LOOKUP( $S, n$ ), which fetches the type of member  $n$  for type  $S$ . An instantiation of LOOKUP for TypeScript is provided by Bierman et al. [10].

**Statements and Type Environments.** The typing rules for statements are presented in Figure 6. Type environments are modified by statements, in particular variable declarations and function definitions. We use the notation  $\Gamma_1 \vdash s \rightarrow \Gamma_2$  to indicate that after executing statement  $s$  in type environment  $\Gamma_1$ , the new environment is  $\Gamma_2$ .

Variable declarations introduce the identifier with declared type into the type environment, provided the identifier is not already defined (DECL). The type environment defines the context to evaluate expressions (EXPR) and return statements (RET). Return statements are only well-typed inside function bodies. The statements inside statement blocks and if-then-else statements must

$$\begin{array}{c}
\text{[LIT-NUM]} \frac{}{\Gamma \vdash \backslash d+ : \text{number}} \quad \text{[LIT-STR]} \frac{}{\Gamma \vdash "\backslash w*" : \text{string}} \quad \text{[LIT-BOOL]} \frac{}{\Gamma \vdash \text{true, false} : \text{boolean}} \\
\text{[IDENT]} \frac{(x : T) \in \Gamma}{\Gamma \vdash x : T} \quad \text{[ANON]} \frac{\Gamma \cup \bar{p} \vdash e : T}{\Gamma \vdash (\bar{p}) \Rightarrow e : (\bar{p}) \Rightarrow T} \quad \text{[CALL]} \frac{\Gamma \vdash f : (\bar{x} : \bar{S}) \Rightarrow T \quad \Gamma \vdash \bar{e} : \bar{S}}{\Gamma \vdash f(\bar{e}) : T} \\
\text{[GROUP]} \frac{\Gamma \vdash e : T}{\Gamma \vdash (e) : T} \quad \text{[OP]} \frac{\Gamma \vdash e_1 : S_1 \quad \Gamma \vdash e_2 : S_2 \quad S_1 \odot S_2 : T}{\Gamma \vdash e_1 \odot e_2 : T} \quad \text{[MEM]} \frac{\Gamma \vdash e : S \quad \text{LOOKUP}(S, n) = T}{\Gamma \vdash e.n : T}
\end{array}$$

Figure 5. Typing rules for  $L_B$ 's expressions.

$$\begin{array}{c}
\text{[DECL]} \frac{x \notin \Gamma}{\Gamma \vdash \text{let } x : T; \succ \rightarrow \Gamma \cup (x : T)} \quad \text{[EXPR]} \frac{\Gamma \vdash e : T}{\Gamma \vdash e; \succ \rightarrow \Gamma} \quad \text{[RET]} \frac{\text{inside function body } \Gamma \vdash e : T}{\Gamma \vdash \text{return } e; \succ \rightarrow \Gamma} \\
\text{[BLOCK]} \frac{\Gamma \vdash \bar{s}_B \succ \rightarrow \Gamma_B}{\Gamma \vdash \{\bar{s}_B\} \succ \rightarrow \Gamma} \quad \text{[FUN]} \frac{x \notin \Gamma \quad \Gamma \cup (x : (\bar{p}) \Rightarrow T) \cup (\bar{p}) \vdash \bar{s}_x \succ \rightarrow \Gamma_x}{\Gamma_1 \vdash \text{function } x (\bar{p}) : T \{ \bar{s}_x \} \succ \rightarrow \Gamma \cup (x : (\bar{p}) \Rightarrow T)} \\
\text{[ITE]} \frac{\Gamma \vdash s_{if} \succ \rightarrow \Gamma_{if} \quad \Gamma \vdash s_{else} \succ \rightarrow \Gamma_{else}}{\Gamma \vdash \text{if } (e) s_{if} \text{ else } s_{else} \succ \rightarrow \Gamma} \quad \text{[NOP]} \frac{}{\Gamma \vdash \bullet \succ \rightarrow \Gamma} \quad \text{[SEQ]} \frac{\Gamma_1 \vdash \bar{s} \succ \rightarrow \Gamma_2 \quad \Gamma_2 \vdash s \succ \rightarrow \Gamma_3}{\Gamma_1 \vdash \bar{s} s \succ \rightarrow \Gamma_3}
\end{array}$$

Figure 6. Type environment extension rules for sequences of statements in  $L_B$ .

$$\begin{array}{c}
\text{[R-DECL]} \frac{\Gamma \vdash \bar{s} : R}{\Gamma \vdash \text{let } x : T; \bar{s} : R} \quad \text{[R-EXPR]} \frac{\Gamma \vdash \bar{s} : R}{\Gamma \vdash e; \bar{s} : R} \quad \text{[R-RET]} \frac{\Gamma \vdash e : R}{\Gamma \vdash \text{return } e; \bar{s} : R} \\
\text{[R-BLOCK-SELF]} \frac{\Gamma \vdash \bar{s}_B : R \quad \Gamma \vdash \bar{s}}{\Gamma \vdash \{\bar{s}_B\} \bar{s} : R} \quad \text{[R-BLOCK-NEXT]} \frac{\Gamma \vdash \bar{s}_B \quad \Gamma \vdash \bar{s} : R}{\Gamma \vdash \{\bar{s}_B\} \bar{s} : R} \\
\text{[R-FUN]} \frac{\Gamma \cup (x : (\bar{p} \Rightarrow R)) \vdash \bar{s} : R' \quad \Gamma \cup (x : (\bar{p}) \Rightarrow R) \cup (\bar{p}) \vdash \bar{s}_x : R}{\Gamma \vdash \text{function } x (\bar{p}) : R \{ \bar{s}_x \} \bar{s} : R'} \\
\text{[R-ITE-SELF]} \frac{\Gamma \vdash s_{if} : R \quad \Gamma \vdash s_{else} : R}{\Gamma \vdash \text{if } (e) s_{if} \text{ else } s_{else} \bar{s} : R} \quad \text{[R-ITE-NEXT]} \frac{\Gamma \vdash \bar{s} : R}{\Gamma \vdash \text{if } (e) s_{if} \text{ else } s_{else} \bar{s} : R}
\end{array}$$

Figure 7.  $L_B$ 's typing rules for function returns.

maintain valid type environments, but do not have an external effect (BLOCK, ITE). This also applies to function definitions; however, the defined function is finally added to the external type environment (FUN). Lastly, empty statements do not alter the type environment (NOP), while statement sequences propagate the type environment along the execution (SEQ).

**Return Types.** The rules for checking return types are presented in Figure 7. Firstly, return statements must contain expressions matching the function's declared return type. Secondly, such an expression must be returned on every execution path. We use the notation  $\Gamma \vdash \bar{s} : R$  to indicate the sequence of statements  $\bar{s}$  ensures a return value of type  $R$ .

For variable declarations and expression statements, the return type of the subsequent statements is considered (R-DECL, R-EXPR). The return type of a return statement directly corresponds to the type of the returned expression (R-RET). For statement blocks, the return type is decided by either the block itself or the subsequent statements (R-BLOCK-SELF, R-BLOCK-NEXT). In function definitions, the



return type is determined by the type of the subsequent statements, similar to expression statements. It is additionally required that the function body returns a type matching the declared return type (R-FUN). For if-then-else statements, both branches must return the same type (R-ITE-SELF), or the return type is determined by the following statements (R-ITE-NEXT).

**Language Definition.** In summary, a program  $s$  is in language  $L_B$  if both (i)  $s$  conforms to the grammar in Figure 4 and (ii)  $s$  is well-typed according to the typing rules in Figures 5–7.

### 3.2 Prefix Automaton Definition

We introduce a general definition of *prefix automata*, beginning with basic automata concepts. Prefix automata are standard automata that ensure a special *prefix property*<sup>3</sup>. This property enables us to use a prefix automaton to decide whether some string is in the prefix language  $L^P$  of a given language  $L$ . That is, the prefix automaton can function as a completion engine  $CE_L$  to facilitate constrained decoding, as described in §2.

We consider an automaton  $A := \langle \Sigma, Q, \delta, I, F \rangle$ , a tuple of the five following elements: (i)  $\Sigma$  is an alphabet of input symbols; (ii)  $Q$  is a set of states; (iii)  $\delta : Q \times \Sigma \mapsto \mathcal{P}(Q)$  is a computable transition function that maps a state and an input symbol to a finite set of next states; (iv)  $I \subseteq Q$  is a finite set of initial states; and (v)  $F \subseteq Q$  is a decidable set of accepting states. As a convention, we denote a symbol in  $\Sigma$  as  $c$ , a string of symbols in  $\Sigma^*$  as  $s$ , the empty string as  $\varepsilon$  and an operator for concatenating symbols and strings as  $\circ$ . The transition function  $\delta$  maps a given state to all possible subsequent states. When  $\delta$  is applied on a set of states  $\mathbf{q} \subseteq Q$ , we take the union of the results as output, i.e.,  $\delta(\mathbf{q}, c) := \bigcup_{q \in \mathbf{q}} \delta(q, c)$ . The transition function defines a directed graph  $G$  over  $Q$ , where every state is a node and there is an edge annotated with  $c$  from  $q$  to  $q'$  if  $q' \in \delta(q, c)$ . The language parsed by  $A$  comprises all strings  $s$  such that traversing  $G$  from some initial state in  $I$  along the edges annotated with  $c_i$  for  $c_1 \circ c_2 \circ \dots \circ c_n = s$ , it is possible to reach some accepting state in  $F$ . Formally, we define recursively a traversal function  $\gamma$  for states  $\mathbf{q}$  as  $\gamma(\mathbf{q}, \varepsilon) := \mathbf{q}$  and  $\gamma(\mathbf{q}, s \circ c) := \delta(\gamma(\mathbf{q}, s), c)$ . The language accepted by  $A$  is then defined as  $L(A) := \{s \mid \gamma(I, s) \cap F \neq \emptyset\}$ . The traversal function has two intuitive properties concerning reachability that can be shown inductively:

(P1) A path along the graph can be split arbitrarily, i.e.,  $\gamma(\mathbf{q}, s \circ s') = \gamma(\gamma(\mathbf{q}, s), s')$ .

(P2) If a state is reached by  $s \circ s'$ , some state is reachable by  $s$ , i.e.,  $\gamma(\mathbf{q}, s \circ s') \neq \emptyset \implies \gamma(\mathbf{q}, s) \neq \emptyset$ .

An automaton satisfies the *prefix property* or is a *prefix automaton*, if there is a path from every reachable state to some accepting state, or formally:

**DEFINITION 2.** For an automaton  $A$ , the *prefix property* holds iff  $\forall q \in \gamma(I, s) : \exists s' : \gamma(q, s') \cap F \neq \emptyset$ . The automaton is a *prefix automaton* if it satisfies the *prefix property*.

Intuitively, for such automata, reaching some state by consuming string  $s$  implies that  $s$  is a prefix to some member of  $L(A)$ . We define the reachable language of  $A$ , all inputs that result in some state, as  $L_r(A) := \{s \mid \gamma(I, s) \neq \emptyset\}$ . Below, we establish the equivalence of  $L_r(A)$  and  $L(A)^P$ , the prefix language of  $L(A)$  as defined in Definition 1.

**LEMMA 1.** If  $A$  is a *prefix automaton*, then  $L(A)^P = L_r(A)$ .

**PROOF.** For any  $s \in L(A)^P$  there exists  $s'$  such that  $s \circ s' \in L(A)$ , by the definition of prefix languages. By the definition of  $L(A)$ , this implies  $\gamma(I, s \circ s') \neq \emptyset$ . Then, using (P2), we further derive  $\gamma(I, s) \neq \emptyset$ , i.e.,  $s \in L_r(A)$ . Therefore,  $L(A)^P \subseteq L_r(A)$  holds. The other direction also holds. We first see that  $s \in L_r(A) \implies \gamma(I, s) \neq \emptyset$ . Then applying Definition 2 and (P1), we find  $\exists s' : \gamma(I, s \circ s') \cap F \neq \emptyset$ , implying  $s \circ s' \in L(A)$  and thus  $s \in L(A)^P$ .  $\square$

<sup>3</sup>Note that the prefix property defined in our work differs from the one discussed in classical texts, e.g., [31]

Note that  $L(A)^P \subseteq L_r(A)$  holds generally for automata, since the first half of the proof does not require the prefix property.

**From Prefix Automata to Completion Engines.** With Lemma 1, given a prefix automaton  $A$ , we can define a convenient-to-compute completion engine for the underlying language  $L(A)$ :  $CE_{L(A)}(s) := \gamma(I, s) \neq \emptyset$ . Since our target language is  $L$  and not  $L(A)$ , we now need to determine the relationship between  $L(A)$  and  $L$ . If we construct  $A$  such that it parses a subset of  $L$ , i.e.,  $L(A) \subseteq L$ , we are guaranteed that all LLM generations constrained by  $CE_{L(A)}$  lie in  $L$ . Conversely, if  $L(A) \supseteq L$ , we are guaranteed that every string in  $L$  can be expressed under constrained decoding, but not that every generation is valid. For example, if  $A$  permits all syntactically correct programs, it guarantees that all well-typed programs can be generated, but permits ill-typed programs as well. Therefore,  $L(A) \subseteq L$  is required to achieve our goal of enforcing well-typedness on LLM-generated code. Ideally,  $A$  would parse  $L$  exactly, i.e.,  $L(A) = L$ , which in our setting additionally guarantees that every well-typed program can be expressed under the constraints of the completion engine. If this is not achieved, it is important for  $A$  to capture a large subset of  $L$  to be practically useful.

**Building a Prefix Automaton for  $L_B$ : Warming up.** In the next sections, we will construct a prefix automaton for soundly parsing well-typed programs in  $L_B$ , by presenting various prefix automata for well-typed fragments of  $L_B$ . Our final automaton will cover a significant but incomplete subset of  $L_B$ . Incompleteness exists because to ensure that our algorithms terminate, we do not cover high-order types that are less likely to occur in practice. This is discussed in more detail in §3.4. Our evaluation in §5 empirically demonstrates that our approach sufficiently covers practical use cases to significantly improve the correctness of LLM-generated code.

We choose  $\Sigma$  to be the set of Unicode characters. This makes our completion engine agnostic to LLM vocabularies. Even though LLMs' vocabularies differ, their tokens are always a string of single or multiple characters. When our completion engine for  $L_B$  is called during constrained decoding, i.e., at Line 6 of Algorithm 1, it processes the sampled token character by character.

Before proceeding, we briefly introduce several base prefix automata below, with their precise definitions detailed in the Appendix. These automata are later combined, with parts of the transition function being overwritten, to construct more complex automata that capture elements of  $L_B$ .

- Union  $A_X \cup A_Y$  parses the language  $\{s \mid s \in L(A_X) \cup L(A_Y)\}$ . It is a prefix automaton if both  $A_X$  and  $A_Y$  are prefix automata.
- Concatenation  $A_{XY}$  parses the language  $\{s \circ s' \mid s \in L(A_X), s' \in L(A_Y)\}$ . It is a prefix automaton if  $A_X$  and  $A_Y$  are both prefix automata, and  $L(A_Y) \neq \emptyset$ .
- Kleene-Star  $A_{\bar{X}}$  parses the language  $\{\bar{s} \mid s \in L(A_X)\}$ . It is a prefix automaton if  $A_X$  is a prefix automaton.
- Terminal  $A_S$  parses the language  $\{S\}$ , where  $S$  is a fixed, non-empty string.
- Empty  $A_\emptyset$  parses the empty language  $\emptyset$  and is always a prefix automaton.

### 3.3 Prefix Automata for Identifiers, Literals, and Types

We now introduce prefix automata for basic syntactic elements of  $L_B$ : identifiers, literals, and type annotations. The languages parsed by these automata exactly match their counterparts in  $L_B$ .

**Literals.** The prefix automaton for literals  $A_l := A_{\text{NUM}} \cup A_{\text{STR}} \cup A_{\text{BOOL}}$  accepts number, string, and boolean literals as defined in Figure 4. The automata  $A_{\text{NUM}}$ ,  $A_{\text{STR}}$ , and  $A_{\text{BOOL}}$  are defined by the deterministic finite automaton representation of the corresponding regular expression of the literal. To ensure the prefix property on the finite automata of the regular expression, we prune states from which accepting states can not be reached.

**Identifiers.** During parsing, we maintain the current type environment  $\Gamma$ , as detailed in §3.5. We define the identifier automaton  $A_x$  as the union of the terminal automata for identifiers defined in  $\Gamma$ . In other words,  $A_x := \bigcup_{y \in \Gamma} A_y$ .

**Types.** The type automaton  $A_T$  accepts type annotations as defined in the grammar of  $L_B$  (Figure 4). It is defined as  $A_T := A_{\text{TYPE-LIT}} \cup A_{\text{TYPE-FUN}}$ . This includes type literal automaton  $A_{\text{TYPE-LIT}} := A_{\text{string}} \cup A_{\text{number}} \cup A_{\text{boolean}}$  and function type automaton  $A_{\text{TYPE-FUN}} := A_{(\bar{p}) \Rightarrow T}$ . The latter is a concatenation of multiple prefix automata, with the parameter and return types recursing on  $A_T$ . This recursive definition is valid, since it ensures a finite set of initial states, defines a decidable accepting set, and preserves the prefix property.

### 3.4 Prefix Automaton for Expressions

We introduce prefix automata to parse well-typed expressions in  $L_B$ . We begin by describing an automaton  $A_e$  to parse expressions whose types are unrestricted, e.g., any expression  $e$  in an expression statement  $e;$ . Then, we present an automaton  $A_e \downarrow T$  for expressions whose type is constrained to  $T$ , e.g., for parameters of function calls. The type-constrained version accepts a string only if the inhabited type of the represented expression matches  $T$ . To preserve the prefix property, we need to ensure that partial expressions can be completed to inhabit the constrained type. Completions may involve arbitrarily many applications of operators, which may modify the expression type. We therefore introduce a type search algorithm that soundly determines which types an expression can inhabit, and use it to prune transitions that violate the prefix property.

**Unrestricted Expressions.** To handle the recursive syntactic structure of expressions, we differentiate two kinds as shown in Figure 4: *base expressions*, including identifiers, literals, grouped expressions, and anonymous functions, and *extension expressions*, which are operator applications (binary operator, member access, or function call) that lead to extending a given expression.

The expression automaton  $A_e$  is thus defined as the union of base expression automata  $A_x$ ,  $A_l$ ,  $A_{(e)}$ , and  $A_{(\bar{p}) \Rightarrow e}$ , with potential extensions  $A_{\odot e}$ ,  $A_{.n}$ , and  $A_{(\bar{e})}$ . The individual base and extension automata are constructed by concatenating the respective terminal automata and recursively  $A_e$ . Additionally, we restrict the type of the recursive  $A_e$  if the restriction is required by the type system, e.g., for parsing call parameters with a fixed type. We provide additional details on this restriction in the Appendix. Since an expression can end after either base or extensions, accepting states of both base and extending automata are accepting states of  $A_e$ . To implement extensions, we start from the base expression automata and recursively adjust  $A_e$ 's transition function  $\delta_e$  by adding outgoing edges from the accepting states of the current automaton to the initial states of the extending automata, or formally:

$$\forall X, Y : \delta_e(q_Y^X, c) := \begin{cases} \delta_Y(q_Y^X, c) \cup \delta_e(I_{(\bar{e})}^X, c) \cup \delta_e(I_{\odot e}^X, c) \cup \delta_e(I_{.n}^X, c) & \text{if } q_Y^X \in F_Y \\ \delta_Y(q_Y^X, c) & \text{otherwise,} \end{cases}$$

where the labels  $X$  and  $Y$  for a state  $q_Y^X$  represent that a string  $X$  has been parsed, and currently the active automaton is  $A_Y$ , which can be one of the following:  $A_x$ ,  $A_l$ ,  $A_{(e)}$ ,  $A_{(\bar{p}) \Rightarrow e}$ ,  $A_{\odot e}$ ,  $A_{.n}$ , and  $A_{(\bar{e})}$ . The superscripts are useful for tracking the currently expressed type, enabling us to determine the validity of extensions and transition to type-restricted expressions based on  $L_B$ 's typing rules. For instance, for state  $q^{42}$ , the addition operator extension  $+e$  and function call extension  $(\bar{e})$  are syntactically applicable to 42 of type number. While the addition operator with type signature  $\text{number} + \text{number} : \text{number}$  is allowed, we can not apply a function call on number. In general, we set  $I_Y^X := \emptyset$  when  $Y$  is an invalid extension to  $X$ . Moreover, for the extension  $+e$  to be valid,  $e$  must be of type number. To this end, we transition to a type-restricted expression automaton by setting

$I_{+e}^{42}$  to the set of initial states for  $A_+ \circ (A_e \downarrow \text{number})$ . Similar to the recursive type automaton, our definition of  $A_e$  ensures a finite set of initial states and a decidable accepting set.

**Type-Constrained Expressions.** To implement  $A_e \downarrow T$ , we must determine whether a partial expression  $s$  can be completed to inhabit type  $T$ . Completing  $s$  without any extension can lead to a possible set of types and repeated extensions can further alter the result type, but we are not guaranteed that the desired type can be reached. Moreover, extensions can be applied indefinitely, prohibiting an exhaustive search of possible completions.

We therefore develop a two-tiered algorithm, which we describe in the following paragraphs. This algorithm first identifies the *derivable types*  $\text{DERIVABLE}(q_s)$  of  $s$  based on its current state  $q_s$ .  $\text{DERIVABLE}(q_s)$  refers to the set of inhabitable types for all possible expressions completed from  $s$  without extension. Second, a *type reachability search*  $\text{REACHABLE}(\text{DERIVABLE}(q_e), T)$  is performed to determine if  $T$  can be inhabited by extending from the derivable types of  $s$ .

We prune automaton transitions when this type search returns a negative result. To ensure the prefix property, the performed search is *sound*, i.e., it only returns a positive result if  $T$  can be expressed by a valid sequence of extensions. This also aligns with our goal of generating only well-typed programs, ensuring that our expression automata accept a subset of all well-typed expressions of  $L_B$ . To ensure termination, the search is *incomplete*, i.e., there may be a valid sequence of transitions to express  $T$  which is not found by the search and we may end up disallowing generation of a well-typed expression. However, it only avoids traversing types of high complexity that are less likely to occur in practice. We further empirically ensure that our approach is practically effective (§5).

**Derivable Types.** For the first part of the algorithm, we determine all types inhabitable by the currently parsed expression  $s$  without extension, i.e.,  $\text{DERIVABLE}(q_s)$ . For example, while parsing partial identifier  $x$  in the type environment  $\Gamma := \{(x : \text{number}), (xy : \text{string})\}$ , we have  $\text{DERIVABLE}(q_x) = \{\text{number}, \text{string}\}$  and  $\text{DERIVABLE}(q_{xy}) = \{\text{string}\}$ . For a final state  $q$  of expression  $e$ , we define  $\text{DERIVABLE}(q) := T$ , where  $\Gamma \vdash e : T$ . Different expressions impose different

Table 1. Definition of  $\text{DERIVABLE}(x)$  for partial expressions introduced in Figure 4.  $s \leq s'$  expresses that  $s$  is a prefix of  $s'$ .  $\text{pmatch}(s, T)$  determines whether a prefix  $s$  partially matches the regular expression of literals of type  $T$ .

| $s$                       | $\text{DERIVABLE}(q_s)$                                                                  |
|---------------------------|------------------------------------------------------------------------------------------|
| $l$                       | $\{T \mid \text{pmatch}(l, T), T \in \{\text{number}, \text{string}, \text{boolean}\}\}$ |
| $x$                       | $\{T \mid x \leq n, (n : T) \in \Gamma\}$                                                |
| $(\bar{p}) \Rightarrow e$ | $\{(\bar{p}) \Rightarrow T \mid \text{REACHABLE}(\text{DERIVABLE}(q_e), T)\}$            |
| $(e$                      | $\{T \mid \text{REACHABLE}(\text{DERIVABLE}(q_e), T)\}$                                  |
| $e \odot$                 | $\{T \mid \exists S' : \Gamma \vdash e : S \wedge S \odot S' : T\}$                      |
| $e($                      | $\{R \mid \Gamma \vdash e : (\bar{p}) \Rightarrow R\}$                                   |
| $e.a$                     | $\{S \mid a \leq n, \Gamma \vdash e : T, \text{LOOKUP}(T, n) = S\}$                      |

rules on derivability, and we present the detailed rules in Table 1. Note that for grouped expressions and function literals, we need to enumerate reachable types by recursively contained expressions. To avoid explicitly enumerating all reachable types, we integrate the derivability and reachability algorithms. This optimization is discussed in more detail in the Appendix.

**LEMMA 2.** For state  $q \in \gamma(I_e, s)$  of partial expression  $s$ ,  $\text{DERIVABLE}(q)$  returns all  $T$  s.t. exists some suffix  $s'$  with  $\Gamma \vdash s \circ s' : T$  and  $s'$  does not involve an extension (operator, call, or member access).

**PROOF.** By case distinction on the possible states of partial expressions.  $\square$

**Type Reachability.** To determine which types are inhabitable by extending a base expression  $e$  of a given type  $T$  (with binary operator, function call, or member access), we analyze sequences of single extension steps with compatible signatures. This process is conceptualized as a search over a graph where types are nodes and extension steps are edges. For every binary operator  $\odot$

**Algorithm 2** Our type reachability search algorithm**Input:** Current type  $T$  of some expression  $e$ , goal type  $G$ **Output:** Whether  $G$  can be reached by extending  $e$ 


---

```

1: function REACHABLE( $T, G$ )
2:   if  $T = G$  then return true                                ▶ The goal type is successfully found
3:   if  $T$  is marked then return false else mark  $T$              ▶ Type  $T$  is marked to avoid cycles
4:   for each valid extension step  $\diamond$  from  $T$  do
5:      $S :=$  the resulting type of applying  $\diamond$  on  $T$ 
6:     if PRUNESearch( $T, G, S$ ) continue                        ▶ Prune the search to ensure termination
7:     if REACHABLE( $S, G$ ) return true                            ▶ Recurse to the next round of extension
8:   return false                                                ▶ No suitable extension is found

```

---

with the signature  $T \odot X : S$ , an edge is created from type  $T$  to type  $S$ . As an example, the operator for numerical addition  $+$  has the signature  $\text{number} + \text{number} : \text{number}$ , thereby forming an edge from  $\text{number}$  to itself. Furthermore, for every member  $n$  of type  $T$ , we create an edge from  $T$  to  $\text{LOOKUP}(T, n)$ , e.g., from  $\text{number}$  to  $() \Rightarrow \text{string}$  for the member  $\text{toString}$  of  $\text{number}$  type. Finally, we connect each function type  $(\bar{p}) \Rightarrow R$  and with its return type  $R$ . For instance,  $() \Rightarrow \text{string}$  is connected with  $\text{string}$ . Examples of type graphs can be found in §2.2 and Figure 3. Note that these extension steps are *abstract*, in the sense that they focus on the type of the expression being extended and the resulting type after extension, not considering textual representation and parameters.

The type reachability algorithm, Algorithm 2, implements a depth-first search over this type graph, starting from the current type  $T$ , succeeding upon finding goal type  $G$  (Line 2), marking any visited types to prevent cycles (Line 3). Then, it proceeds to iterate over all valid extension steps from  $T$  (Line 4) and computes the resulting type  $S$  after the extension step is applied (Line 5). In the conceptualized type graph, as described in the previous paragraph, this is equivalent to exploring all outgoing edges from  $T$ . At Line 7, we proceed to recursively search if  $S$  can reach  $G$ . If all recursive calls are unsuccessful, the goal type can not be reached (Line 8).

Some programming languages define self-referential default members, e.g., `clone` in Java or `valueOf` in TypeScript, which are nullary functions that return a value of the same type as the callee,  $() \Rightarrow T$  for type  $T$ . When these members are accessed in functions, higher-order functions can be derived indefinitely. For instance, for a function  $f$  with type  $() \Rightarrow S$ ,  $f.\text{valueOf}$  has the type  $() \Rightarrow () \Rightarrow S$ . We therefore need to restrict the type search to a finite set of types to ensure termination. At Line 6 of Algorithm 2, we add a heuristic `PRUNESearch` into the search, which decides where to prune the search process. We develop a simple heuristic based on the results from Gvero et al. [30]. This heuristic prunes exploration of types with higher complexity than goal or source type if they do not contain yet unexplored primitive types, thus preventing exploration of arbitrarily complex types. The details of this heuristic are presented in the Appendix. While ensuring termination, our heuristic leads to incompleteness and the potential rejection of well-typed expressions. However, this effect is less pronounced in practical usage, as only highly complex (thus less realistically used) types are avoided.

We proceed to prove the soundness of Algorithm 2 below.

**LEMMA 3.** *The type search in Algorithm 2 is sound, i.e., for any expression  $e$  with  $\Gamma \vdash e : T$ , if  $\text{REACHABLE}(T, G)$  holds, then there exists a sequence of extensions  $y$  such that  $\Gamma \vdash e \circ y : G$ .*

**PROOF.** By the design of Algorithm 2, if  $\text{REACHABLE}(T, G)$  returns true, there is a sequence of  $n$  recursive calls to  $\text{REACHABLE}(T_i, G)$ , with  $T_0 = T$  and  $\text{REACHABLE}(T_n, G) = \text{true}$ . Each  $T_i$  ( $i > 0$ ) is derived because some extension  $\diamond_i$  is applicable to  $T_{i-1}$  based on the typing rules of  $L_B$ . We then

convert each  $\diamond_i$  to its concrete, textual version  $\blacklozenge_i$ . This representation includes the required well-typed parameters of  $\blacklozenge_i$  (i.e., for binary operators and non-nullary functions), which are constructed using literals. Finally, we construct  $y$  as  $\blacklozenge_1 \circ \dots \circ \blacklozenge_n$ .  $\square$

Note that using any pruning heuristic at Line 6 of Algorithm 2 preserves soundness, which in turn is sufficient to preserve the required prefix property, as defined in Definition 2. We can conclude that the two-tiered search algorithm soundly determines whether the desired target type can be derived from some partial input. Therefore, we conclude that  $A_e \downarrow T$  and  $A_e$  are prefix automata that parse a subset of well-typed expressions in  $L_B$ .

**COROLLARY 4.** *If  $\text{REACHABLE}(\text{DERIVABLE}(q), G)$  holds for any  $q \in \gamma(I_e, s)$  of a partial expression  $s$ , then there exists a suffix  $s'$  such that  $\Gamma \vdash s \circ s' : G$ .*

**PROOF.** This conclusion follows directly from Lemmas 2 and 3.  $\square$

**LEMMA 5.** *The language parsed by  $A_e \downarrow T$  is thus a subset of the expressions of  $L_B$  of type  $T$ , i.e.,  $L(A_e \downarrow T) \subseteq \{s \mid \Gamma \vdash s : T\}$ . Since  $A_e$  recursively involves  $A_e \downarrow T$ , the language parsed by  $A_e$  is also a subset of well-typed expressions of  $L_B$ , i.e.,  $L(A_e) \subseteq \{s \mid \exists T : \Gamma \vdash s : T\}$ .*

### 3.5 Prefix Automata for Statements

We define the remaining automata to capture the complete language  $L_B$ . The statement automaton is defined recursively as  $A_s := A_{\text{DECL}} \cup A_{\text{EXPR}} \cup A_{\text{RET}} \cup A_{\text{BLOCK}} \cup A_{\text{FUN}} \cup A_{\text{ITE}}$ . The declaration automaton  $A_{\text{DECL}} := A_{\text{let } x : T}$ ; captures undefined variable names  $x$  by accepting all strings, except for existing identifiers. This automaton is a prefix automaton since an accepting state can always be reached by appending characters to the declared identifier. The return statement automaton is  $A_{\emptyset}$  when outside a function and restricts the parsed expression to the return type of the surrounding function otherwise. The remaining automata are mainly concatenations of previously defined automata and recursive invocations of  $A_s$ , with small variations detailed in the Appendix.

**Tracking Type Environments.** Generally, we follow the typing rules in Figure 6. Identifiers are passed on through all state transitions, matching the rule  $\text{SEQ}$ , where the type environment of consecutive statements needs to be compatible. However, in the cases of  $\text{BLOCK}$ ,  $\text{ITE}$  and  $\text{FUN}$ , we discard the local type environment after parsing, matching the respective typing rules. In  $\text{FUN}$  additionally, the function signature and parameters are added into the type environment of the function body automaton, and the function signature in the environment of subsequent statements.

**Guaranteeing Return Types.** When parsing the body of a function, the transition function of the function automata  $A_{\text{FUN}}$  maintains information about the declared return type and the encountered return statements (if any).  $A_{\text{FUN}}$  only accepts states where all return values match the declared return type and all execution paths inside the function body return, following  $L_B$ 's typing rules in Figure 7. If the current generated statements do not return in all execution paths, another statement is forced to be generated. Since we can always express the requested type through literals, a correct return statement can always be generated and the prefix automaton property is not violated.

The described rules are straightforward to implement without violating the prefix property as all restrictions are derived only from already parsed input, e.g., the already defined identifiers or the previously declared function return type. We can therefore deduce that the statement automaton is a prefix automaton. Moreover, the automaton accepts all valid statements of  $L_B$ , with the exception of well-typed expressions rejected by  $A_e$ . Therefore the parsed language is a subset of  $L_B$ .

**LEMMA 6.** *With  $A_M := A_{\bar{s}}$  it holds that  $A_M$  is a prefix automaton and  $L(A_M) \subseteq L_B$*



#### 4 Extension to TypeScript

We extend our completion engine described in §3 to handle a core subset of modern TypeScript. In this section, we selectively discuss the implementation of several interesting TypeScript features. We provide a comprehensive list of supported and unsupported TypeScript features in the Appendix.

**Constant Variable Declarations.** In addition to variable declaration using `let`, TypeScript supports constant declarations using `const`. This defines immutable identifiers. We thus additionally track mutability of each identifier in the type environment and disallow applying the assignment operator to immutable identifiers.

**Arrays.** We add support for array type annotation, parsing array expressions, and reading from and assigning to array fields. In array expressions, we enforce that all array elements have the same type. Moreover, array types introduce another dimension of type nesting. Therefore we adapt the type reachability pruning heuristic to handle this additional dimension to ensure termination.

**Loops.** TypeScript supports various loop constructs, including `for`, `while`, `do-while`, and `for...of` loops. These are implemented mostly as variations of the statement block parser. The `for...of` loop uniquely constrains the right-hand side of the `...of` operator to an array of any type. To adapt the type search, we introduce a generic array type  $\bullet[]$ , which matches any array type. For example, both types `number[]` and `string[]` match  $\bullet[]$  in Line 2 of Algorithm 2.

**Additional Operators and Types.** We add several arithmetic and logic operators, such as modulo `%`, exact equality `===`, logical or `||`, and the ternary operator `?:`. To handle these operators, we add additional edges to the type search graph. Moreover, we add support for post- and prefix operators such as `--` and `++`, which are only valid extensions to mutable expressions.

**Operator Precedence.** TypeScript defines an operator precedence, which determines the implicit grouping of expressions. For example `1 + 2.toString()` is parsed as `1 + (2.toString())`. We adapt our expression parsing algorithm in two places to handle operator precedences. First, in the expression automaton, we leverage the knowledge about previously parsed extensions to determine the implicit grouping and thus where the next operator is applied. For example, for state  $q^{1+2}$ , the member access extension `.n` is applied to 2, as opposed to `1 + 2`. Second, we adapt the type search in Algorithm 2. Concretely, we ensure that only extensions that can be validly applied based on operator precedence are iterated over. For this, we track the operator precedence of previously parsed extensions and extensions considered during the traversal of the type graph and omit operators in Line 5 that violate operator precedence.

**Global Identifiers and Imports.** In TypeScript, many identifiers are defined globally and available in any execution. These global identifiers are incorporated by initializing the type environment of the program automaton accordingly. Identifiers such as `Math` introduce additional types, which we additionally implement. We also model the import of the `crypto` library using `require`.

**Polymorphic Built-In Members.** The TypeScript LOOKUP implementation defines a few polymorphic members for built-in types. For example, for array `x` of type `T[]`, `x.map(f)` takes a callback function `f` and returns a new array `[f(x[0]), f(x[1]), ...]`. If `f` has type `(T) => P`, the returned array has type `P[]`. Here `P` is a *type parameter*, which is instantiated by matching the type of the passed function to the type pattern.

We support such polymorphisms by adapting the type search. We track type patterns and enforce that type parameters are instantiated before the goal type is reached. We then continue the search from the instantiated version. In the `map` example, when searching completions of `x.map`, we first search for functions that instantiate the type parameter, and then continue the search from the

instantiated type. When anonymous functions are generated as call parameters, we enforce that the function matches the searched type pattern.

**Type Annotations.** TypeScript is designed to be flexible, allowing many type annotations to be omitted when they can be automatically inferred. We generally support this, such as inferring types from initial values. However, it can lead to unexpected types when annotations are omitted, often confusing even experienced developers [47, 48]. Moreover, in the context of LLM-based code generation, having more type annotations can provide valuable information for both the model and our type-constraining algorithms. We have identified three situations where generated code often fails to compile without type annotations, prompting us to enforce them. First, we require annotations for all function parameters and return types. Second, all variable declarations must either have a type annotation or be initialized with a value. Third, we enforce type annotations for the first parameter of anonymous functions used as callbacks in the polymorphic built-in member `reduce`. These constraints trade off practical correctness with theoretical language completeness.

## 5 Experimental Evaluation

We present an extensive evaluation of our type constraining approach on a variety of tasks and models. We outline our experimental setup (§5.1), evaluate the impact on compilation errors and functional correctness (§5.2), perform runtime analysis (§5.3), and present case studies (§5.4).

### 5.1 Experimental Setup

We now outline our main evaluation setup, covering implementation, evaluated tasks, considered models, compared methods, and metrics. We provide further setup details and hyperparameter choices in the Appendix.

**Implementation.** Our implementation is written in Python and contains 11249 lines of code. To ensure robust implementation, we built a large set of around four hundred unit tests and frequently compared the behaviors of our implementation with the official TypeScript compiler [42].

**Tasks and Benchmarks.** We evaluate three relevant tasks of code generation:

- *Synthesis*: Given a natural language task description and a function header, the task is to generate a solution from scratch.
- *Translation*: Given a function written in Python and the header of an equivalent TypeScript function, the task is to generate the body of the equivalent function in TypeScript.
- *Repair*: Given a natural language task description, a non-compilable solution, the corresponding compiler error, and the function header, the task is to restore functionality of the flawed solution by resolving the compilation error.

The benchmarks for these tasks are based on TypeScript-translated tasks from HumanEval [12] and MBPP [5], contained in the MultiPL-E dataset [13], with 159 and 384 instances each. We observe that success in generating valid code for the same sample can vary depending on the random seed used. To obtain more comprehensive results on the small HumanEval dataset, we generate each sample 4 times with different seeds and aggregate the outcomes. In MBPP, we generate each sample once. For Repair, we collect all non-compiling programs from the unconstrained synthesis task for all models, resulting in 292 and 248 instances for HumanEval and MBPP each.

**Models.** We use 6 different open-weight LLMs, covering 3 LLMs of varying parameter sizes from the same model family and 4 models of a similar size from different model families: the Gemma 2 model family with 2B/9B/27B parameters [63], DeepSeek Coder 33B (abbreviated as DS Coder 33B) [28], CodeLlama 34B [58], and Qwen2.5 32B [72]. For all evaluated LLMs we choose the

Table 2. Number of instances with compiler errors in unconstrained generation (Vanilla), idealized syntax-only constraining (Syntax), and our proposed type constraining (Types). Type constraining reduces compiler errors by 74.8% and 56.0% in the synthesis of HumanEval and MBPP problems respectively, compared to only 9.0% and 4.8% ideal improvement on the two datasets respectively through syntax-only constraining.

|           | Model         | Synthesis |          |                  | Translation |           |                  | Repair  |          |                   |
|-----------|---------------|-----------|----------|------------------|-------------|-----------|------------------|---------|----------|-------------------|
|           |               | Vanilla   | Syntax   | Types            | Vanilla     | Syntax    | Types            | Vanilla | Syntax   | Types             |
| HumanEval | Gemma 2 2B    | 103       | 92↓10.7% | <b>44</b> ↓57.3% | 177         | 149↓15.8% | <b>80</b> ↓54.8% | 194     | 181↓6.7% | <b>103</b> ↓46.9% |
|           | Gemma 2 9B    | 45        | 41↓8.9%  | <b>13</b> ↓71.1% | 75          | 63↓16.0%  | <b>16</b> ↓78.7% | 113     | 108↓4.4% | <b>52</b> ↓54.0%  |
|           | Gemma 2 27B   | 15        | 13↓13.3% | <b>2</b> ↓86.7%  | 20          | 20↓0.0%   | <b>3</b> ↓85.0%  | 45      | 40↓11.1% | <b>22</b> ↓51.1%  |
|           | DS Coder 33B  | 26        | 25↓3.8%  | <b>5</b> ↓80.8%  | 18          | 17↓5.6%   | <b>7</b> ↓61.1%  | 36      | 36↓0.0%  | <b>15</b> ↓58.3%  |
|           | CodeLlama 34B | 86        | 71↓17.4% | <b>28</b> ↓67.4% | 158         | 124↓21.5% | <b>59</b> ↓62.7% | 153     | 142↓7.2% | <b>48</b> ↓68.6%  |
|           | Qwen2.5 32B   | 17        | 17↓0.0%  | <b>2</b> ↓88.2%  | 24          | 21↓12.5%  | <b>5</b> ↓79.2%  | 36      | 34↓5.6%  | <b>13</b> ↓63.9%  |
| MBPP      | Gemma 2 2B    | 67        | 64↓4.5%  | <b>27</b> ↓59.7% | 126         | 111↓11.9% | <b>79</b> ↓37.3% | 194     | 184↓5.2% | <b>108</b> ↓44.3% |
|           | Gemma 2 9B    | 30        | 29↓3.3%  | <b>10</b> ↓66.7% | 67          | 61↓9.0%   | <b>33</b> ↓50.7% | 129     | 124↓3.9% | <b>63</b> ↓51.2%  |
|           | Gemma 2 27B   | 20        | 19↓5.0%  | <b>7</b> ↓65.0%  | 37          | 36↓2.7%   | <b>22</b> ↓40.5% | 71      | 69↓2.8%  | <b>32</b> ↓54.9%  |
|           | DS Coder 33B  | 32        | 32↓0.0%  | <b>19</b> ↓40.6% | 29          | 27↓6.9%   | <b>13</b> ↓55.2% | 90      | 90↓0.0%  | <b>43</b> ↓52.2%  |
|           | CodeLlama 34B | 80        | 71↓11.2% | <b>41</b> ↓48.8% | 126         | 114↓9.5%  | <b>54</b> ↓57.1% | 157     | 148↓5.7% | <b>76</b> ↓51.6%  |
|           | Qwen2.5 32B   | 19        | 18↓5.3%  | <b>13</b> ↓31.6% | 22          | 22↓0.0%   | <b>16</b> ↓27.3% | 55      | 52↓5.5%  | <b>29</b> ↓47.3%  |

instruction-tuned variants, which are fine-tuned to follow instructions in a chat-style interaction, such that they adequately attempt to resolve the presented tasks.

**Compared Methods.** We run unconstrained LLM sampling, reported as *Vanilla*. We measure the upper bound improvement of prior syntactic constraining methods [8, 56, 65] by assuming that all syntactically incorrect instances generated by Vanilla could be compiled under syntactic constraining. We refer to this improvement as idealized *Syntax*. We separately sample using type-constrained decoding based on our completion engine introduced in §3 and §4, and report it as *Types*. Due to the size and complexity of the full TypeScript compiler, featuring over 427,105 lines of code in 698 files [42], our extension does not cover all features of TypeScript. We therefore emulate a type constraining that supports the entire TypeScript feature set. Concretely, if a sample compiles correctly without any constraining, we report it as-is. Otherwise, we report the result of a constrained resample. For all methods, if generation takes more than 300 seconds, we report the partial program generated until the timeout.

**Metrics.** We compute two main metrics to assess the effectiveness of the compared methods. First, we determine the number of *compiler errors* in model-generated outputs. We count as a compiler error any case in which the TypeScript compiler [42] reports an issue during compilation. To measure *functional correctness*, we leverage the pass@1 metric [14], which measures the percentage of code generations that pass the provided unit tests given only one trial.

## 5.2 Results on Compilation and Functional Correctness

In this section, we present our experimental results, showing that on all three code-generation-related tasks, our type constraining approach significantly improves the considered LLMs in generating both compilable and functionally correct code. It also substantially outperforms syntax-only constraining.

**Reduction of Compilation Errors.** In Table 2, we present the number of compilation errors produced by each compared method. For synthesis and translation, in the unconstrained setting

Table 3. pass@1 of unconstrained generation (Vanilla) and type constraining (Types). The benefit of our type-constraining approach transfers from reduced compilation errors to improved functional correctness.

|           | Model         | Synthesis |                   | Translation |                    | Repair  |                    |
|-----------|---------------|-----------|-------------------|-------------|--------------------|---------|--------------------|
|           |               | Vanilla   | Types             | Vanilla     | Types              | Vanilla | Types              |
| HumanEval | Gemma 2 2B    | 29.1      | <b>30.2</b> ↑3.8% | 50.2        | <b>53.9</b> ↑7.5%  | 11.6    | <b>20.9</b> ↑79.4% |
|           | Gemma 2 9B    | 56.6      | <b>58.3</b> ↑3.1% | 73.7        | <b>78.3</b> ↑6.2%  | 24.0    | <b>34.9</b> ↑45.7% |
|           | Gemma 2 27B   | 69.5      | <b>71.2</b> ↑2.5% | 86.6        | <b>87.7</b> ↑1.3%  | 38.4    | <b>41.1</b> ↑7.1%  |
|           | DS Coder 33B  | 68.9      | <b>71.1</b> ↑3.2% | 88.7        | <b>90.1</b> ↑1.6%  | 47.6    | <b>50.7</b> ↑6.5%  |
|           | CodeLlama 34B | 41.0      | <b>43.4</b> ↑5.7% | 58.6        | <b>63.5</b> ↑8.3%  | 17.5    | <b>27.4</b> ↑56.9% |
|           | Qwen2.5 32B   | 79.6      | <b>81.8</b> ↑2.8% | 92.1        | <b>93.9</b> ↑1.9%  | 65.4    | <b>71.2</b> ↑8.9%  |
| MBPP      | Gemma 2 2B    | 40.4      | <b>42.4</b> ↑5.2% | 52.3        | <b>56.0</b> ↑7.0%  | 12.1    | <b>22.6</b> ↑86.7% |
|           | Gemma 2 9B    | 65.4      | <b>67.4</b> ↑3.2% | 71.4        | <b>75.8</b> ↑6.2%  | 24.2    | <b>31.9</b> ↑31.7% |
|           | Gemma 2 27B   | 70.6      | <b>72.1</b> ↑2.2% | 83.1        | <b>84.4</b> ↑1.6%  | 39.1    | <b>45.2</b> ↑15.5% |
|           | DS Coder 33B  | 65.4      | <b>67.2</b> ↑2.8% | 85.9        | <b>89.1</b> ↑3.6%  | 35.1    | <b>43.1</b> ↑23.0% |
|           | CodeLlama 34B | 42.2      | <b>45.6</b> ↑8.0% | 55.7        | <b>63.3</b> ↑13.6% | 15.7    | <b>26.6</b> ↑69.2% |
|           | Qwen2.5 32B   | 76.3      | <b>76.6</b> ↑0.3% | 89.6        | <b>90.4</b> ↑0.9%  | 48.0    | <b>54.0</b> ↑12.6% |

(Vanilla), on average only 9.0% and 4.9% of the non-compiling instances in HumanEval and MBPP respectively are due to syntactic errors (Syntax), with Qwen2.5 32B even making no syntax errors at all for HumanEval synthesis and MBPP translation. In contrast, type constraining reduces compilation errors by more than half, i.e., by 75.3% and 52.1% on HumanEval and MBPP respectively. We observe that models across all sizes and families benefit similarly from our constraining, with a minimum error reduction of 54.8% and 27.3% on HumanEval and MBPP respectively, highlighting the general effectiveness of our approach.

A straightforward way to improve successful compilation of LLM-generated code is to feed the erroneous code and the error message back to an LLM for correction — our repair task. Thanks to its general applicability, our type constraining approach can also enhance this process. Our experimental results in the setting of code repair are also depicted in Table 2. We find that, in the vanilla setting, many models struggle to correctly localize and resolve compilation errors, with Gemma 2 2B for example repairing only 33.5% and 25.8% of the non-compiling HumanEval and MBPP instances, respectively. This is substantially increased to 56.4% and 58.4% through type constraining. On average, using type-constrained sampling, 53.7% more compilation errors are resolved than using vanilla LLM decoding.

**Improving Functional Correctness.** Programs that do not compile are always functionally incorrect. With our type constraining method, non-compilable generations can be turned into well-formed ones, offering the possibility of achieving functional correctness. In Table 3, we experimentally show that type constraining universally improves the functional correctness of LLM-generated code. On the three tasks considered, employing type constraining improves LLMs’ pass@1 rate, achieving an average increase by 3.5% in synthesis, 5.0% in translation, and 37.0% in repair tasks. The larger improvement in the latter is due to vanilla LLMs generally struggling to generate functionally correct code. One interesting phenomenon is that, for stronger models, constraints more likely lead to recovering functionally correct code. For example on the synthesis task, for Gemma 2 27B, out of the 26 instances that required resampling to compile successfully, 17 are also functionally correct. For Qwen2.5 32B, 15 out of 21 such instances were correct.

### 5.3 Runtime Analysis

As discussed in §2, compared with vanilla LLM decoding, our constrained decoding algorithm runs an additional loop (Line 4 of Algorithm 1), where tokens are sampled from an LLM-produced next-token probability distribution and checked against the completion engine. In this section, we investigate how this process introduces additional runtime overhead for our type constraining. Note that for each selected token, vanilla and constrained decoding both run LLM inference only once, meaning that there is no extra overhead from LLM inference in constrained decoding.

**Overhead of Type Constraining.** For an application of our method in practice, the effective runtime increase due to constrained decoding is highly relevant. To assess it, we measure the runtime per synthesis instance in HumanEval and MBPP for both unconstrained and type-constrained decoding. We report the median runtime per instance for type constraining and its relative increase to unconstrained decoding in Table 4. On average over the evaluated models, we observe a relative increase of 39.1% and 52.1% in HumanEval and MBPP respectively. We consider this impact to be bearable for the observed significant decrease in compilation errors. Moreover, this is measured on an unoptimized, Python-based implementation and could be significantly improved by a more system-oriented implementation, such as the one proposed by Dong et al. [18].

Table 4. Median time per synthesis instance in seconds spent by our type-constrained decoding and its relative increase compared with unconstrained decoding (Vanilla).

| Model         | HumanEval  | MBPP       |
|---------------|------------|------------|
| Gemma 2 2B    | 6.7↑38.3%  | 6.3↑35.4%  |
| Gemma 2 9B    | 8.3↑29.2%  | 9.5↑46.8%  |
| Gemma 2 27B   | 11.7↑19.9% | 11.7↑32.8% |
| DS Coder 33B  | 11.5↑36.2% | 9.4↑59.5%  |
| CodeLlama 34B | 7.6↑40.8%  | 7.0↑37.6%  |
| Qwen2.5 32B   | 7.3↑39.6%  | 4.9↑54.8%  |

**Number of Sample-and-Check Loop Iterations.** To provide an in-depth analysis of the overhead of our type constraining method, we measure the number of iterations spent by the sample-and-check loop to find an admissible token. The results are provided in Figure 8. We observe that the number of loop iterations follows a long-tail distribution. For 99.4% of cases, only one loop iteration is needed. This number is even higher for stronger models, with Gemma 2 9B and 27B requiring one iteration in 99.6% and 99.9% of cases, respectively. This means that, in most instances, LLMs can generate a valid token on the first attempt, which is then verified by the completion engine. In cases where more than one iteration is needed, the completion engine intervenes to guide the selection of valid tokens. These interventions help resolve errors in many instances in our benchmarks, providing significant benefit, as discussed in §5.2.

Prior work [8, 56, 65] implemented constrained decoding differently than Algorithm 1. Instead of running the sample-and-check loop, they execute the completion engine for all tokens in the LLM’s vocabulary, mask out all invalid tokens, and sample once from the remaining valid tokens based on their normalized likelihoods. This implementation is less efficient than ours, especially when calling the completion engine is costly. Based on the long-tail distribution depicted in Figure 8, our implementation requires only a single invocation to the completion engine for most cases, whereas their implementation calls it for every token in the LLM’s vocabulary for all cases. In our experiments, their implementation results in timeouts for all benchmark instances, with a timeout of 5 minutes per instance. This justifies our choice of using the sample-and-check loop.

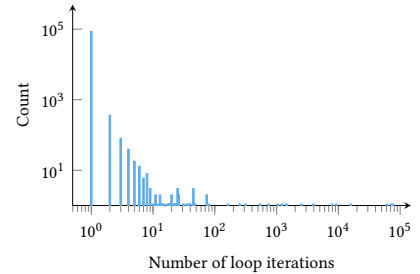


Figure 8. Histogram on the number of iterations consumed by the sample-and-check loop at Line 4 of Algorithm 1 to find a valid token, measured with Gemma 2 2B for HumanEval synthesis.

Table 5. Three examples illustrating the effect of type-constrained sampling. Left are unconstrained generations with problematic tokens highlighted in **red**, and right are type-constrained results with corrected tokens highlighted in **green**, adapted for clarity. In (a), Gemma 2 2B attempts to call `split`, missing required arguments. In (b), DeepSeek Coder 33B attempts to complete a function without a guaranteed return. The issue is resolved by forcing generation of another statement after the main loop. In (c), Gemma 2 9B calls `reduce` with an anonymous function without type annotation. This leads to an incorrect type inference for the first parameter. The issue is solved by guiding the model to add type annotation.

|     | Vanilla                                                                                                                                                      | Types                                                                                                                                                                 |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| (a) | <pre>// find all words with consonants function select_words(s: string): string[] {   for (const word of s. <b>split()</b>)</pre>                            | <pre>// find all words with consonants function select_words(s: string): string[] {   for (const word of s. <b>split(' ')</b>)</pre>                                  |
| (b) | <pre>function largest_divisor(n: number): number {   for(let i = n - 1; i &gt; 0; i--) {     if(n % i === 0) return i;   }   <b>// No return</b> }</pre>     | <pre>function largest_divisor(n: number): number {   for(let i = n - 1; i &gt; 0; i--) {     if(n % i === 0) return i;   }   <b>return 1;</b> }</pre>                 |
| (c) | <pre>// insert a delimiter between every element return nums.reduce(   (<b>acc</b>, curr, index) =&gt; {     acc.push(curr);     return acc;   }, []);</pre> | <pre>// insert a delimiter between every element return nums.reduce(   (<b>acc: number[]</b>, num, index) =&gt; {     acc.push(curr);     return acc;   }, []);</pre> |

## 5.4 Case Study

For a qualitative evaluation, we manually inspect instances where unconstrained decoding fails and our type constraining approach successfully corrects errors. We find that our technique effectively amends various types of compilation errors. We showcase three such examples in Table 5.

**Wrong Parameter Count.** In Table 5a, the task is to find all words in a string that contain a number of consonants. When Gemma 2 2B attempts to solve this problem, translating from the Python version to TypeScript, it calls `split` without arguments. This is allowed in Python, but not in TypeScript, where a single string argument is required. Type constraining correctly detects this and guides the generation to sample a space character as parameter.

**Missing Return Statement.** In Table 5b, to complete function `largest_divisor`, the model must compute a straightforward divisor loop. DeepSeek Coder 33B Instruct [28] implements a correct loop, but does not guarantee returning a value in every execution path. When the return statement in the loop is never executed, e.g., for negative inputs, the function thus returns undefined, violating the type rules. Our method detects this issue and forces the generation of another statement in the function body, resulting in a correct fallback return statement.

**Incorrect Type Inference.** In Table 5c, the task is to insert a delimiter between every element in an array. Gemma 2 9B solves this with the `reduce` function. This generic function accepts two arguments; first, a callback function that is called consecutively for every element in the array and accumulates a result, second, an initial value for the callback function. The type of the accumulator of the callback is derived implicitly from the second argument, which is an empty array in the given example. TypeScript infers special type `never[]` for the empty array, disallowing inserting `curr` of type `number` through `push`. Therefore, the program fails to compile. This issue is a well-known limitation of the TypeScript compiler, often confusing even expert developers [47, 48]. Our method resolves it by enforcing adequate type annotation on the first argument of the callback function.



```

1 function sort_third(l: number[], r: number[]): number[] {
2   for (let i = 0; i < l.length; i++) {
3     r.push(l[i].toString().slice(0, 3).concat(l[i].toString().slice(3).split('').reverse())
4     .join('')).split('').reverse().join('').toString() + l[i].toString().slice(3).split('') ...

```

Figure 9. Compilation errors remain when the model does not terminate after a corrected token. In this example for synthesis on the HumanEval task #33, CodeLlama 34B is steered away from accessing non-existing member `.sort` and instead accesses `.toString`.

## 6 Discussion

Our general type constraining approach, backed by strong experimental results, opens exciting avenues for future research, which we discuss below.

**Remaining Compiler Errors.** We observe that, even though constrained decoding guarantees a valid result upon termination, a considerable amount of compilation errors remain due to non-termination within the token or time limit. We find this to be caused by generation loops, entered when generation is amended by constraints and the LLM is unable to recover. An example is depicted in Figure 9, where CodeLlama 34B tries to access the invalid member `sort` on an expression of type `number`. Future work may add additional constraints to force stopping such unconstructive loops and steer the model more strictly, e.g., by limiting the complexity of generated expressions.

**Implementation Effort.** Developing a completion engine for a target programming language currently requires manual efforts. However, we expect that the involved effort to adopt our method to other languages will be reduced significantly, as many features transfer from our implementation for  $L_B$  and TypeScript. Moreover, we believe that, due to the huge impact on LLM’s code generation, the effort will pay off. Future programming language developers may consider generally writing their compilers as an incremental completion engine, which additionally enables automatic adoption for constrained code generation, besides conventional grammar parsing and type checking.

**Broader Application to More Complex Tasks and Stronger LLMs.** Stronger LLMs, such as the latest OpenAI models [33], may make fewer typing errors on the HumanEval and MBPP datasets. Our evaluation results in Table 2 also demonstrate that compilation errors decrease with increasing model size for the Gemma family. However, recent findings showed that currently, even the strongest LLMs struggle with generating compilable code for more complex coding tasks, stricter typing rules, and low-resource languages (e.g., new DSLs). Gusanidas [29] evaluated various state-of-the-art LLMs on difficult code synthesis tasks in Rust, reporting compilation error rates of 18% for OpenAI o1-mini [33], 39% for DeepSeek R1 [15] and 27% for Anthropic’s Claude 3.5 Sonnet [2]. For OCaml and Haskell, which are sparsely represented in LLMs’ training data, the error rate is even higher at 40% – 60% for all models, matching a trend of worse performance on low-resource languages [24, 36]. Pan et al. [53] compiled a large dataset of code translation and found 44.3% of GPT-4-generated code to contain compilation errors. Similarly, Shetty et al. [60] report around 25% compilation errors for C-to-Rust translation using OpenAI o1 models. Our type constraining approach is broadly applicable to all these scenarios and our work presents a promising proof of concept. Future work can consider building upon our approach to address these challenges.

Constrained decoding in general requires access to the next-token probability distributions produced by LLMs. Currently, commercially available black-box LLM APIs only return sampled tokens and do not offer complete next-token distributions. A possible solution is to integrate our method into the backend of model providers, as was recently implemented for guaranteeing adherence to JSON Schemas [3, 50].

## 7 Related Work

**Code Language Models.** Recently, LLMs have gained traction for diverse coding tasks such as code synthesis, repair, or translation [35]. These models are typically trained on datasets containing billions to trillions of tokens and have billions of parameters, with both factors contributing to improved performance in code-related benchmarks [28, 46, 58, 63]. Meanwhile, LLMs are well known to frequently make mistakes [32, 57], and, as we show in this work, even state-of-the-art open-weight models with over 30 billion parameters frequently make errors in code generation.

**Improving Language Model Accuracy.** Apart from constrained decoding, three primary approaches have been proposed to enhance the accuracy of language models on code tasks: fine-tuning, retrieval augmentation (RAG), and compiler or execution feedback. Fine-tuning adapts the model weights based on specifically collected training data. This process is highly resource intensive [64, 69]. RAG provides the model with additional context based on a database or related code snippets [6, 56]. Compiler and execution feedback is only available after completing the model generation and requires resampling [16, 34, 68]. However, constrained decoding is orthogonal to these methods and, as indicated by Poesia et al. [56] and our experimental results, combining constrained decoding with RAG or compiler feedback additionally improves model performance.

**Constrained Decoding.** Prior work on constrained decoding failed to achieve strong results due to its limitation to syntactic language features. Constraining to context-free languages has been explored extensively in recent work [7, 8, 56, 70]. Simple context-sensitive syntactic features, such as the space indentation in Python and the scope markers in Go have also been implemented [41, 65]. As demonstrated in §5, however, syntax errors on average account for only 6% of compilation errors in recent code models. The rarity of syntax errors significantly reduces the potential of leveraging them for improvements in code correctness. Meanwhile, our type-constrained decoding more than halved compilation errors.

**Type Systems for Code Synthesis.** Previous work that leveraged type systems for code synthesis was confined to specialized settings and unable to constrain general, complex program generation. Poesia et al. [56] proposed using known column names to guide SQL query generation. Gvero et al. [30] employed a search on the type graph for function call completion. Agrawal et al. [1] leverage language-server-generated type annotations for object member accesses. Blinn et al. [11] use language-server-derived type information to provide additional context to the LLM, but not to enforce hard constraints. Additionally, type constraints have been used to direct code synthesis based on specialized search procedures [22, 55, 68]. However, these methods are not compatible with LLM-based code generation. This limits their ability to exploit the powerful natural language and general-purpose capabilities of LLMs.

## 8 Conclusion

In this work, we explored how type systems in programming languages can be used to guide language models during decoding. Concretely, we design and implement prefix automata to perform type constraining for a foundational simply typed language and then extend it to the popular language TypeScript. We extensively evaluate the impact of using such constraints for code synthesis, translation, and repair and observe that we more than halve compilation errors on a diverse set of models and consistently increase functional correctness. We further explore qualitatively how the constraining positively impacts code generation. We conclude that such type constraining should be implemented for more programming languages, and has the potential to generally improve code generation in many domains.

## Artifact Availability

The artifact for this paper, including source code, datasets, and reproductions scripts, is available on GitHub (<https://github.com/eth-sri/type-constrained-code-generation>) and Zenodo [45].

## Acknowledgements

We would like to thank the anonymous reviewers for their in-depth and constructive feedback, and the artifact reviewers for their feedback on our artifact accessibility.

## References

- [1] Lakshya Agrawal, Aditya Kanade, Navin Goyal, Shuvendu K Lahiri, and Sriram Rajamani. 2023. Monitor-Guided Decoding of Code LMs with Static Analysis of Repository Context. In *NeurIPS*. <https://openreview.net/forum?id=qPUbKxKvXq>
- [2] Anthropic. [n. d.]. Claude 3 Model Card. <https://assets.anthropic.com/m/61e7d27f8c8f5919/original/Claude-3-Model-Card.pdf> Accessed: March 10, 2025.
- [3] Anthropic. 2025. JSON Mode. <https://docs.anthropic.com/en/docs/build-with-claude/tool-use#json-mode> Accessed: March 10, 2025.
- [4] Ken Arnold and James Gosling. 1996. *The Java Programming Language*.
- [5] Jacob Austin, Augustus Odena, Maxwell I. Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, et al. 2021. Program Synthesis with Large Language Models. *arXiv Preprint* (2021). <https://arxiv.org/abs/2108.07732>
- [6] Nastaran Bassamzadeh and Chhaya Methani. 2024. A Comparative Study of DSL Code Generation: Fine-Tuning vs. Optimized Retrieval Augmentation. *arXiv Preprint* (2024). <https://doi.org/10.48550/arXiv.2407.02742>
- [7] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2023. Prompting Is Programming: A Query Language for Large Language Models. *PLDI* (2023). <https://doi.org/10.1145/3591300>
- [8] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2024. Guiding LLMs The Right Way: Fast, Non-Invasive Constrained Generation. In *ICML*. <https://openreview.net/forum?id=pXaEYzrFae>
- [9] Satwik Bhattamishra, Kabir Ahuja, and Navin Goyal. 2020. On the Ability and Limitations of Transformers to Recognize Formal Languages. In *EMNLP*. <https://doi.org/10.18653/v1/2020.emnlp-main.576>
- [10] Gavin M. Bierman, Martin Abadi, and Mads Torgersen. 2014. Understanding TypeScript. In *ECOOP*.
- [11] Andrew Blinn, Xiang Li, June Hyung Kim, and Cyrus Omar. 2024. Statically Contextualizing Large Language Models with Typed Holes. *OOPSLA* (2024). <https://doi.org/10.1145/3689728>
- [12] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language Models are Few-Shot Learners. In *NeurIPS*. <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>
- [13] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q. Feldman, et al. 2023. MultiPL-E: A Scalable and Polyglot Approach to Benchmarking Neural Code Generation. *IEEE Trans. Software Eng.* (2023).
- [14] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv Preprint* (2021). <https://arxiv.org/abs/2107.03374>
- [15] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, et al. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv Preprint* (2025). <https://doi.org/10.48550/arXiv.2501.12948>
- [16] Pantazis Deligiannis, Akash Lal, Nikita Mehrotra, Rishi Poddar, and Aseem Rastogi. 2025. RustAssistant: Using LLMs to Fix Compilation Errors in Rust Code. In *ICSE*. <https://www.microsoft.com/en-us/research/publication/rustassistant-using-llms-to-fix-compilation-errors-in-rust-code/>
- [17] TypeScript Developers. [n. d.]. TypeScript: Documentation – More on Functions. <https://www.typescriptlang.org/docs/handbook/2/functions.html#function-type-expressions> Accessed: March 10, 2025.
- [18] Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. 2024. XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models. *arXiv Preprint* (2024). <https://doi.org/10.48550/arXiv.2411.15100>
- [19] Alan AA Donovan and Brian W Kernighan. 2015. *The Go programming language*.
- [20] Shihan Dou, Haoxiang Jia, Shenxi Wu, Huiyuan Zheng, Weikang Zhou, Muling Wu, Mingxu Chai, Jessica Fan, Caishuang Huang, Yunbo Tao, et al. 2024. What’s Wrong with Your Code Generated by Large Language Models? An Extensive Study. *arXiv Preprint* (2024). <https://doi.org/10.48550/arXiv.2407.06153>

- [21] Javid Ebrahimi, Dhruv Gelda, and Wei Zhang. 2020. How Can Self-Attention Networks Recognize Dyck-n Languages?. In *EMNLP*. <https://aclanthology.org/2020.findings-emnlp.384/>
- [22] Jonás Fiala, Shachar Itzhaky, Peter Müller, Nadia Polikarpova, and Ilya Sergey. 2023. Leveraging Rust Types for Program Synthesis. *PLDI* (2023). <https://doi.org/10.1145/3591278>
- [23] Zheng Gao, Christian Bird, and Earl T. Barr. 2017. To type or not to type: quantifying detectable bugs in JavaScript. In *ICSE*. <https://doi.org/10.1109/ICSE.2017.75>
- [24] Alessandro Giagnorio, Alberto Martin-Lopez, and Gabriele Bavota. 2025. Enhancing Code Generation for Low-Resource Languages: No Silver Bullet. *arXiv Preprint* (2025). <https://doi.org/10.48550/arXiv.2501.19085>
- [25] GitHub. [n. d.]. <https://github.com/features/copilot>
- [26] GitHub. 2022. The top programming languages. <https://octoverse.github.com/2022/top-programming-languages>
- [27] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The Llama 3 Herd of Models. *ArXiv Preprint* (2024). <https://arxiv.org/abs/2407.21783>
- [28] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming - The Rise of Code Intelligence. *arXiv Preprint* (2024). <https://doi.org/10.48550/arXiv.2401.14196>
- [29] Gusaniadas. [n. d.]. Compilation Benchmark. <https://github.com/Gusanidas/compilation-benchmark> Accessed: March 10, 2025.
- [30] Tihomir Gvero, Viktor Kuncak, Ivan Kuraj, and Ruzica Piskac. 2013. Complete completion using types and weights. In *PLDI*. <https://doi.org/10.1145/2491956.2462192>
- [31] John E. Hopcroft and Jeffrey D. Ullman. 1979. *Introduction to Automata Theory, Languages and Computation*.
- [32] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2023. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *arXiv Preprint* (2023). <https://doi.org/10.48550/arXiv.2311.05232>
- [33] Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. 2024. OpenAI o1 System Card. *arXiv Preprint* (2024). <https://doi.org/10.48550/arXiv.2412.16720>
- [34] Prithwish Jana, Piyush Jha, Haoyang Ju, Gautham Kishore, Aryan Mahajan, and Vijay Ganesh. 2024. CoTran: An LLM-Based Code Translator Using Reinforcement Learning with Feedback from Compiler and Symbolic Execution. In *ECAI (Frontiers in Artificial Intelligence and Applications)*. <https://doi.org/10.3233/FAIA240968>
- [35] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A Survey on Large Language Models for Code Generation. *arXiv Preprint* (2024). <https://doi.org/10.48550/arXiv.2406.00515>
- [36] Sathvik Joel, Jie JW Wu, and Fatemeh H. Fard. 2024. Survey on Code Generation for Low resource and Domain Specific Programming Languages. *arXiv Preprint* (2024). <https://doi.org/10.48550/arXiv.2410.03981>
- [37] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. 2024. StarCoder 2 and The Stack v2: The Next Generation. *arXiv Preprint* (2024). <https://doi.org/10.48550/arXiv.2402.19173>
- [38] Midnight. 2024. GitHut 2.0. [https://madnight.github.io/github/#/pull\\_requests/2024/1](https://madnight.github.io/github/#/pull_requests/2024/1)
- [39] Harry G. Mairson. 2004. Linear lambda calculus and PTIME-completeness. *J. Funct. Program.* (2004). <https://doi.org/10.1017/S0956796804005131>
- [40] Nicholas D Matsakis and Felix S Klock. 2014. The rust language. *ACM SIGAda Ada Letters* (2014).
- [41] Daniel Melcer, Nathan Fulton, Sanjay Krishna Gouda, and Haifeng Qian. 2024. Constrained Decoding for Fill-in-the-Middle Code Language Models via Efficient Left and Right Quotienting of Context-Sensitive Grammars. (2024). <https://arxiv.org/abs/2402.17988>
- [42] Microsoft. 2024. TypeScript. <https://github.com/microsoft/TypeScript>. Accessed on November 9, 2024, commit #ef802b1.
- [43] John C. MITCHELL. 1990. Type Systems for Programming Languages. In *Formal Models and Semantics*. <https://www.sciencedirect.com/science/article/pii/B9780444880741500135>
- [44] Niklas Muennighoff, Qian Liu, Armel Randy Zebaze, Qinkai Zheng, Binyuan Hui, Terry Yue Zhuo, Swayam Singh, Xiangru Tang, Leandro von Werra, and Shayne Longpre. 2024. OctoPack: Instruction Tuning Code Large Language Models. In *ICLR*. <https://openreview.net/forum?id=mw1PWN5WZP>
- [45] Niels Mündler, Jingxuan He, Hao Wang, Koushik Sen, Dawn Song, and Martin Vechev. 2025. *Reproduction Package for "Type-Constrained Code Generation with Language Models"*. [doi:10.5281/zenodo.15355889](https://doi.org/10.5281/zenodo.15355889)
- [46] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. 2024. SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents. In *NeurIPS*. [http://papers.nips.cc/paper\\_files/paper/2024/hash/94f093b41fc2666376fb1f667fe282f3-Abstract-Conference.html](http://papers.nips.cc/paper_files/paper/2024/hash/94f093b41fc2666376fb1f667fe282f3-Abstract-Conference.html)

- [47] nielstron. 2024. Incorrect type deducted for accumulator in reduce. <https://github.com/microsoft/TypeScript/issues/59999>.
- [48] nop33. 2024. Wrong inferred initial value in reduce. <https://github.com/microsoft/TypeScript/issues/59863>.
- [49] OpenAI. 2023. GPT-4 Technical Report. *arXiv Preprint* (2023). <https://doi.org/10.48550/arXiv.2303.08774>
- [50] OpenAI. 2025. Structured Outputs. <https://platform.openai.com/docs/guides/structured-outputs> Accessed: March 10, 2025.
- [51] Gabriel Orlanski, Kefan Xiao, Xavier Garcia, Jeffrey Hui, Joshua Howland, Jonathan Malmaud, Jacob Austin, Rishabh Singh, and Michele Catasta. 2023. Measuring the Impact of Programming Language Distribution. In *ICML*. <https://proceedings.mlr.press/v202/orlanski23a.html>
- [52] Rangeet Pan, Ali Reza Ibrahimzada, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2024. Lost in Translation: A Study of Bugs Introduced by Large Language Models while Translating Code. In *ICSE*. <https://doi.org/10.1145/3597503.3639226>
- [53] Rangeet Pan, Ali Reza Ibrahimzada, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2024. Lost in Translation: A Study of Bugs Introduced by Large Language Models while Translating Code. In *ICSE*. <https://doi.org/10.1145/3597503.3639226>
- [54] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions. In *S&P*. <https://doi.org/10.1109/SP46214.2022.9833571>
- [55] Daniel Perelman, Sumit Gulwani, Thomas Ball, and Dan Grossman. 2012. Type-directed completion of partial expressions. In *PLDI*. <https://doi.org/10.1145/2254064.2254098>
- [56] Gabriel Poesia, Alex Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. 2022. Synchromesh: Reliable Code Generation from Pre-trained Language Models. In *ICLR*. <https://openreview.net/forum?id=KmtVD97J43e>
- [57] Vipula Rawte, Amit P. Sheth, and Amitava Das. 2023. A Survey of Hallucination in Large Foundation Models. *arXiv Preprint* (2023). <https://doi.org/10.48550/arXiv.2309.05922>
- [58] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code Llama: Open Foundation Models for Code. *arXiv Preprint* (2023). <https://doi.org/10.48550/arXiv.2308.12950>
- [59] Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural Machine Translation of Rare Words with Subword Units. In *ACL*. <https://doi.org/10.18653/v1/p16-1162>
- [60] Manish Shetty, Naman Jain, Adwait Godbole, Sanjit A. Seshia, and Koushik Sen. 2024. Syzygy: Dual Code-Test C to (safe) Rust Translation using LLMs and Dynamic Analysis. *arXiv Preprint* (2024). <https://doi.org/10.48550/arXiv.2412.14234>
- [61] Vince Szabo, Dominik Winterer, and Zhendong Su. 2024. Compilation Quotient (CQ): A Metric for the Compilation Hardness of Programming Languages. *arXiv Preprint* (2024). <https://doi.org/10.48550/arXiv.2406.04778>
- [62] Florian Tambon, Arghavan Moradi Dakhel, Amin Nikanjam, Foutse Khomh, Michel C. Desmarais, and Giuliano Antoniol. 2025. Bugs in large language models generated code: an empirical study. *Empir. Softw. Eng.* (2025). <https://doi.org/10.1007/s10664-025-10614-4>
- [63] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. 2024. Gemma 2: Improving Open Language Models at a Practical Size. *arXiv Preprint* (2024). <https://arxiv.org/abs/2408.00118>
- [64] Yun-Da Tsai, Mingjie Liu, and Haoxing Ren. 2024. Code Less, Align More: Efficient LLM Fine-tuning for Code Generation with Data Pruning. (2024). <https://doi.org/10.48550/arXiv.2407.05040>
- [65] Shubham Ugare, Tarun Suresh, Hangoo Kang, Sasa Misailovic, and Gagandeep Singh. 2024. SynCode: LLM Generation with Grammar Augmentation. *ArXiv Preprint* (2024). <https://arxiv.org/abs/2403.01632>
- [66] Pawel Urzyczyn. 1997. Inhabitation in Typed Lambda-Calculi (A Syntactic Approach). In *TLCA (Lecture Notes in Computer Science)*. [https://doi.org/10.1007/3-540-62688-3\\_47](https://doi.org/10.1007/3-540-62688-3_47)
- [67] Heidi Vella. 2024. Google turns to AI to write new code; Workforce reduced. <https://aibusiness.com/data/google-turns-to-ai-to-write-new-code-workforce-reduced>
- [68] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. 2023. Copiloting the Copilots: Fusing Large Language Models with Completion Engines for Automated Program Repair. In *ESEC/FSE*. <https://doi.org/10.1145/3611643.3616271>
- [69] Martin Weyssow, Xin Zhou, Kisub Kim, David Lo, and Houari A. Sahraoui. 2023. Exploring Parameter-Efficient Fine-Tuning Techniques for Code Generation with Large Language Models. *arXiv Preprint* (2023). <https://doi.org/10.48550/arXiv.2308.10462>
- [70] Brandon T. Willard and Rémi Louf. 2023. Efficient Guided Generation for Large Language Models. *arXiv Preprint* (2023). <https://doi.org/10.48550/arXiv.2307.09702>
- [71] Andy Yang, David Chiang, and Dana Angluin. 2024. Masked Hard-Attention Transformers Recognize Exactly the Star-Free Languages. In *NeurIPS*. [http://papers.nips.cc/paper\\_files/paper/2024/hash/13d7f172259b11b230cc5da8768abc5f-](http://papers.nips.cc/paper_files/paper/2024/hash/13d7f172259b11b230cc5da8768abc5f-)

[Abstract-Conference.html](#)

- [72] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024. Qwen2.5 Technical Report. *arXiv Preprint* (2024). <https://doi.org/10.48550/arXiv.2412.15115>
- [73] Qianjun Zhang, Chunrong Fang, Yang Xie, Yuxiang Ma, Weisong Sun, Yun Yang, and Zhenyu Chen. 2024. A Systematic Literature Review on Large Language Models for Automated Program Repair. *arXiv Preprint* (2024). <https://doi.org/10.48550/arXiv.2405.01466>

Received 2024-11-15; accepted 2025-03-06